FIAP ON

DIGITAL BUSINESS ENABLEMENT

WEB SERVICES SOAP (JAX-WS, XML)

2

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 2.1 – As três Perspectivas de Arquitetura SOA.....6
Figura 2.2 – Enterprise Service Bus no centro, conectado a serviços acima e atendendo clientes abaixo.....8
Figura 2.3 – Representação em alto nível do cenário de Solicitação de Crédito 10
Figura 2.4 – Representação em alto nível do cenário de Solicitação de Crédito com a Validação do Cliente em Desenvolvimento na Baixa Plataforma.....11
Figura 2.5 – Representação em alto nível do cenário de Solicitação de Crédito com a Validação do Cliente em Produção na Baixa Plataforma.....12
Figura 2.6 – Representação em alto nível do cenário de obtenção de dados de um cliente não migrado.....13
Figura 2.7 – Tela de criação do projeto OlaClienteWS no Eclipse.....20
Figura 2.8 – Menu de configuração para conversão Maven do projeto OlaClienteWS no Eclipse.....21
Figura 2.9 – Tela de criação do arquivo pom do projeto OlaClienteWS no Eclipse ..21
Figura 2.10 – Resultado da criação do arquivo pom do projeto OlaClienteWS no Eclipse.....21
Figura 2.11 – Tela de criação da classe Cliente no Eclipse.....23
Figura 2.12 – Menu de atalhos no Eclipse 23
Figura 2.13 – Tela de criação da Interface OlaClienteService no Eclipse.....25
Figura 2.14 – Tela de criação da classe OlaClienteServiceImpl no Eclipse.....26
Figura 2.15 – Tela de criação da classe *RunService* no Eclipse.....27
Figura 2.16 – Console exibindo log de execução no Eclipse 28
Figura 2.17 – Browser Chrome exibindo um wsdl local.....28
Figura 2.18 – Tela de criação de um novo projeto no SoapUI 29
Figura 2.19 – Tela de configuração de um novo projeto SoapUI 29
Figura 2.20 – Tela de *request* de um projeto SoapUI.....29
Figura 2.21 – Tela exibindo *request* / response de um projeto SoapUI.....30
Figura 2.22 – Tela de configuração das preferências do CXF no Eclipse.....32
Figura 2.23 – Tela de configuração das preferências do CXF no Eclipse.....32
Figura 2.24 – Tela de criação do projeto OlaClienteConsumer no Eclipse 33
Figura 2.25 – Menu de navegação *New* > *Other* no Eclipse 33
Figura 2.26 – Menu *New* > *Web Service Client* no Eclipse 34
Figura 2.27 – Tela de geração de um *Web Service Client* no Eclipse.....34
Figura 2.28 – Tela de configuração de pacote da geração de um *Web Service Client* no Eclipse.....35
Figura 2.29 – Resultado da geração no Project Explore no Eclipse.....35
Figura 2.30 – Tela de criação da classe OlaClienteMain no Eclipse.....36
Figura 2.31 – Resultado da criação da classe OlaClienteMain no Project Explorer do Eclipse.....36
Figura 2.32 – Menu de navegação *Window* > *Show View* > *Other* no Eclipse.....38
Figura 2.33 – Menu *Show View* > TCP/IP Monitor 38
Figura 2.34 – Tela de configuração *Add* TCP/IP Monitor no Eclipse.....38
Figura 2.35 – Tela de configuração *Start* / *Stop* do TCP/IP Monitor no Eclipse 39
Figura 2.36 – View TCP/IP Monitor do Eclipse com o resultado do chamado ao *endpoint* digaola.....41
Figura 2.37 – Arquitetura de *Web Services* SOAP.....43
Figura 2.38 – UDDI *core data structures*.....44

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 2.39 – Criando um *Dynamic Web Project* para o Exemplo 2 45
Figura 2.40 – Configurando o *Source folder* no *build path* para o Exemplo 2 46
Figura 2.41 – Configurando o *web module* para o Exemplo 2 46
Figura 2.42 – Projeto ClienteCrudSoap no Projeto Explorer Eclipse do Exemplo 2..47
Figura 2.43 – Adicionando o jar Oracle no projeto ClienteCrudSoap do Exemplo 2. 47
Figura 2.44 – Convertendo o projeto ClienteCrudSoap para Maven Project.....47
Figura 2.45 – Criando o arquivo pom do projeto ClienteCrudSoap. 48
Figura 2.46 – O arquivo pom.xml do projeto ClienteCrudSoap no Project Explorer do Eclipse.....48
Figura 2.47 – Criando a classe *EntityManagerFactorySingleton*.....52
Figura 2.48 – Criando a *sequence* SEQ_CLIENTE no Oracle 53
Figura 2.49 – O pacote com.fiap.bank.entity no Project Explorer do Eclipse 53
Figura 2.50 – Log de execução de teste do Hibernate no Console Eclipse 56
Figura 2.51 – Confirmação de gravação no Oracle, via acesso pelo Oracle Developer 56
Figura 2.52 – Criação de pacotes de persistência no Project Explorer do Eclipse....57
Figura 2.53 – Geração de um *Web Service* no wizard do Eclipse.....59
Figura 2.54 – Selecionando a implementação do *Web Service* no *wizard* do Eclipse 60
Figura 2.55 – Configurando um SEI Apache CXF no *wizard* do Eclipse 60
Figura 2.56 – Configurando anotações JAX-WS opcionais no *wizard* do Eclipse.....61
Figura 2.57 – Configurando Java2WS no *wizard* do Eclipse.....61
Figura 2.58 – Resultado da execução o *wizard* no Project Explorer do Eclipse.....62
Figura 2.59 – Criando um novo projeto SOAP no SoapUI 66
Figura 2.60 – Consumindo o método do WS cadastrarCliente no SoapUI.....67
Figura 2.61 – Consumindo o método do WS buscarPorId no SoapUI 67

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 2.1 – *Documento pom*.....23
Código-fonte 2.2 – classe Cliente.....25
Código-fonte 2.3 – interface OlaCliente 25
Código-fonte 2.4 – classe OlaClienteServiceImpl 26
Código-fonte 2.5 – classe OlaClienteServiceImpl 27
Código-fonte 2.6 – classe OlaClienteMain 37
Código-fonte 2.7 – classe OlaClienteMain refatorada.....40
Código-fonte 2.8 – classe OlaClienteMain 41
Código-fonte 2.9 – classe OlaClienteMain 42
Código-fonte 2.10 – pom.xml 50
Código-fonte 2.11 – *persistence*.xml.....51
Código-fonte 2.12 – EntityManagerFactorySingleton.java 52
Código-fonte 2.13 – Cliente.java 55
Código-fonte 2.14 – Teste.java 56
Código Fonte 2.15 – ClienteDAO.java 57
Código-fonte 2.16 – ClienteDaoImpl.java.....57
Código-fonte 2.17 – ClienteServiceImpl.java.....59
Código-fonte 2.18 – ClienteService.java 63
Código-fonte 2.19 – ClienteServiceImpl.java.....63
Código-fonte 2.20 – web.xml.....64
Código-fonte 2.21 – cxf-beans.xml.....65

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

2 WEB SERVICES SOAP (JAX- WS, XML) 6
2.1 SOA - O que é Arquitetura Orientada a Serviços? 6
2.2 *Web Services* 14
2.2.1 Integrando aplicações corporativas, além das redes locais 14
2.2.2 Desenvolvendo *"Big" Web Services* com Apache CXF 17
2.2.2.1 Exemplo 1 - *Web Service standalone* com JAX-WS e CXF 19
2.2.2.2 *Web Service Architecture* e UDDI 42
2.2.3 Exemplo 2 - *Web Service* com JAX-WS, CXF, JPA, Tomcat e Oracle 45
2.3 Considerações Finais 68
REFERÊNCIAS 69

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 WEB SERVICES SOAP (JAX- WS, XML)

2.1 SOA - O que é Arquitetura Orientada a Serviços?

Como você deve ter percebido lendo o resumo deste capítulo, os desafios enfrentados pela TI das grandes corporações são enormes e cada dia mais acelerados pelo mundo de negócios, que está em constante mudança. A adoção de uma abordagem de arquitetura corporativa escalável e flexível, que permita a disponibilização das funcionalidades dos novos sistemas ou sistemas legados por meio de serviços interoperáveis, rapidamente adaptáveis às mudanças de negócio, reutilizáveis e capazes de integrar fornecedores, parceiros e clientes em plataformas heterogêneas é a chave que a TI precisa para manter as grandes empresas vivas e competitivas.

Enfim, podemos afirmar que SOA (*Service-oriented Architecture*) é essa abordagem arquitetural. A Arquitetura Orientada a Serviços é um paradigma para desenvolvimento e manutenção de sistemas distribuídos que suportam importantes processos empresariais de grandes corporações.

Para SOA, existem três importantes perspectivas arquitetônicas, como mostrado na figura: As Três Perspectivas de Arquitetura SOA abaixo:

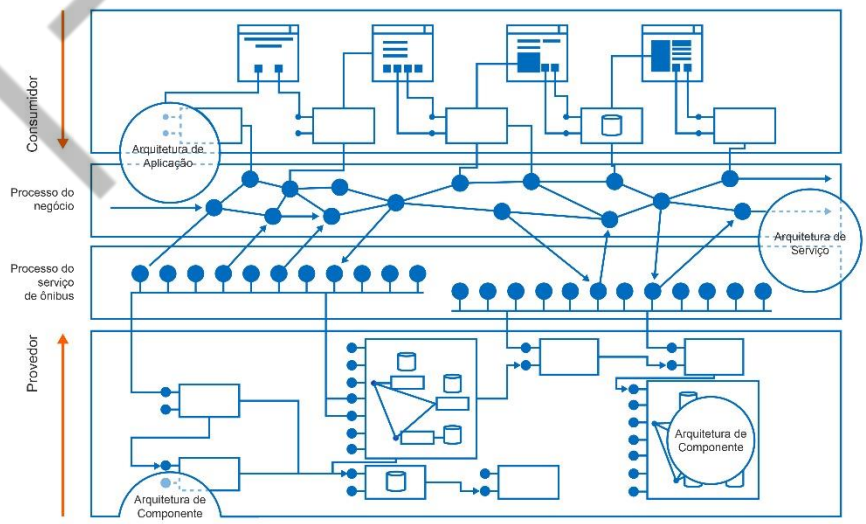

Figura 2.1 – As três Perspectivas de Arquitetura SOA
Fonte: Microsoft, https://msdn.microsoft.com/en-us/library/aa480021.aspx

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- *Application Architecture*: Esta é a face das soluções de negócios que consomem serviços de um ou mais provedores externos e os integra nos processos de negócios da corporação.
- *Service Architecture*: Camada que fornece uma ponte entre os aplicativos consumidores de soluções e as aplicações implementadas internamente. Criando, assim, um barramento de serviços disponíveis para uso, invocados por uma interface comum e com uma arquitetura de gerenciamento providas pelo *Service Bus*.
- *Component Architecture*: Arquitetura composta pelos vários ambientes que suportam as aplicações implementadas internamente, os objetos de negócios corporativos e as respectivas implementações.

A saber, as arquiteturas SOA são desenhadas a partir de três fortes conceitos técnicos:

Utilização de Serviços – Neste contexto, um serviço é a codificação em software de uma funcionalidade que atende a uma necessidade específica do negócio e que, ao ser exposta por meio de uma interface ou contrato, pode ser consumida por requisições feitas por sistemas clientes, internos ou externos. Os serviços podem ser simples, como um serviço que realiza a consulta do saldo bancário de um cliente, recebendo como parâmetros os números da agência e da conta, ou complexos, como um serviço que utiliza um complicado algoritmo de decisão que analisa diversas características do perfil de um cliente para determinar um limite de crédito em dinheiro e o risco de fornecer crédito a ele;
Utilização de ESB (*Enterprise Service Bus*) – "*Enterprise Service Bus*" foi um termo apresentado pelo Gartner em 2002 e utilizado pelo analista Roy Schulte para delinear uma categoria de software que surgiu naquele período. Ainda hoje, existem várias definições sobre os Barramentos de Serviços.

O ESB é um estilo de arquitetura? Uma infraestrutura de serviços ou um padrão de arquitetura?

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O ESB é tudo isso e mais um pouco. Os fornecedores de produtos ESB criaram soluções que possibilitam plugar serviços em um barramento, gerando a infraestrutura necessária para conectar provedores de funcionalidades aos clientes consumidores.

Os sistemas clientes consomem os serviços, enviando mensagens de requisição ao Barramento e não diretamente ao serviço. O Barramento atua como uma camada intermediária, localiza e envia a mensagem ao serviço responsável, recebe uma mensagem com o resultado do processamento do serviço e a encaminha para o cliente solicitante.

Uma solução ESB utiliza um padrão de arquitetura que suporta a organização dos serviços conectados, bem como a orquestração das mensagens em relação ao fluxo de negócio atendido e a interoperabilidade entre ambientes heterogêneos.

Figura 2.2 – Enterprise Service Bus no centro, conectado a serviços acima e atendendo clientes abaixo
Fonte: Oracle Docs:
https://docs.oracle.com/cd/E21764_01/doc.1111/e15020/introduction.htm#OSBCA107

Imagine um cliente em uma revendedora de automóveis, enviando uma solicitação de análise de crédito para a financeira do banco dele, buscando por juros e taxas mais baixos. Simplificando o processo, o cliente utiliza um App da financeira, preenche um formulário e envia a solicitação.

Abstraindo os detalhes de infraestrutura e segurança, a solicitação de crédito chega via Internet ao barramento central de serviços do banco, que é responsável por localizar e chamar o serviço interno de validação do cliente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na sequência, o barramento aciona um segundo serviço, para análise interna de crédito (rendimento do cliente e histórico de financiamentos). Como próximo passo do processo de análise e aprovação de crédito, o barramento chama um terceiro serviço, desta vez externo, para análise de restrições de crédito (SCPC ou Serasa). Por último, invoca o quarto serviço, que consolidará todas as informações processadas pelos serviços anteriores, realizará a persistência da consolidação em um banco de dados e retornará ao App do cliente, a decisão da financeira.

A execução desses passos, seguindo o processo de análise e aprovação de crédito do banco se chama orquestração.

A interoperabilidade está em diversos pontos do cenário descrito acima. Pense comigo: o App é executado em uma plataforma móvel (Android, iOS ou Windows Phone), codificado nativamente ou não. A solução ESB roda na plataforma baixa do banco.

A validação do cliente e a análise interna de crédito, muito provavelmente, utilizam funcionalidades desenvolvidas no mainframe do banco (plataforma alta). A análise de crédito externa é realizada por funcionalidades codificadas de acordo com as plataformas escolhidas pelo SCPC ou Serasa. Ao final, a persistência da consolidação e do resultado da análise é realizada em um SGBD na plataforma baixa do banco (Windows ou Linux).

No cenário acima, podemos pensar facilmente em diversas linguagens de programação, tais como: Android ou Kotlin, Swift, Ionic, Xamarin, React, Angular, Java, .NET, COBOL, CICS, SQL ANSI, PL/SQL, T-SQL, entre outras.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

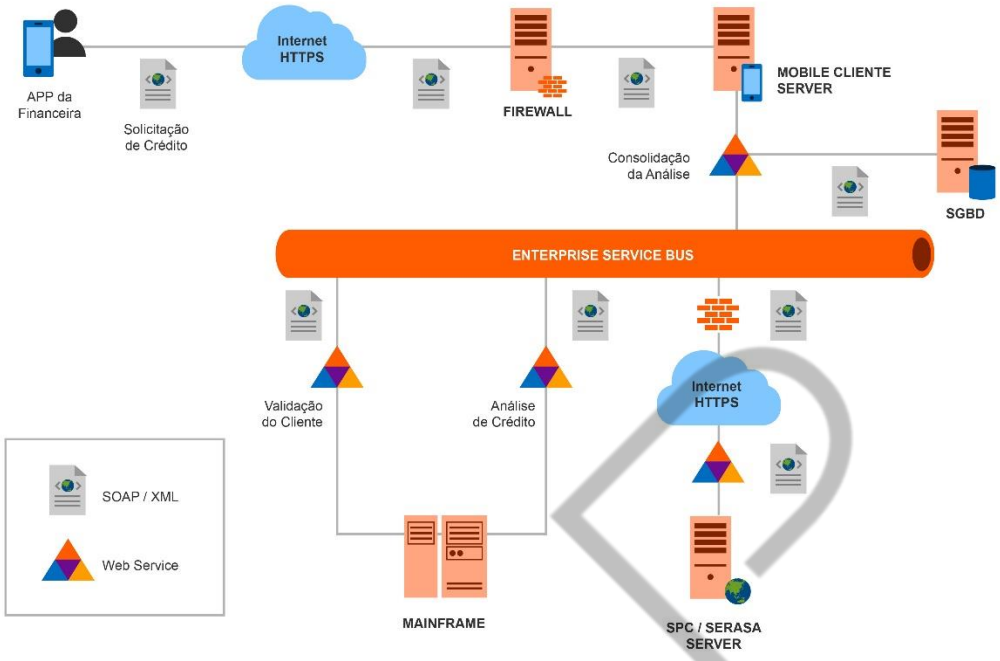

Figura 2.3 – Representação em alto nível do cenário de Solicitação de Crédito
Fonte: Elaborado pelo autor (2017)

Complicado? Para o cliente do banco deve ser simples, sempre!

Ainda sobre o cenário descrito, por que persistir as informações na plataforma baixa do banco, antes de responder ao cliente?

Imagine que, imediatamente após a primeira consulta, o cliente envie uma segunda solicitação de crédito para a financeira. O processo de análise e aprovação do Banco deve ser inteligente o suficiente para possibilitar a parametrização que conduzirá o Barramento e os serviços a aproveitarem as informações geradas no processamento da primeira solicitação, cortando passos na segunda análise, reduzindo custos de valores pagos a provedores externos de análise de restrição de crédito, aliviando o mainframe e respondendo mais rápido ao cliente. Concorda?

Acoplamento Fraco – Ainda sobre o cenário acima, é fácil perceber que o processo de análise e aprovação de crédito utiliza vários "*backends*" para que essa funcionalidade de negócio da financeira seja processada completamente. Também é fácil perceber que o ESB atuando como orquestrador dos serviços, cria uma camada de abstração para o processo da financeira, que não precisa e não deve se preocupar com o *backend* específico que vai resolver o respectivo passo do processo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para exemplificar, imagine que o banco resolveu criar um novo sistema de validação de clientes, agora na plataforma baixa. Isso deveria impactar o App ou o processo de análise e aprovação de crédito da financeira? Claro que não!

Enquanto o novo serviço está em desenvolvimento, o Barramento deve manter a chamada ao serviço de validação de clientes processado pelo mainframe, conforme figura “Representação em alto nível do cenário de Solicitação de Crédito com a Validação do Cliente em Desenvolvimento na Baixa Plataforma”.

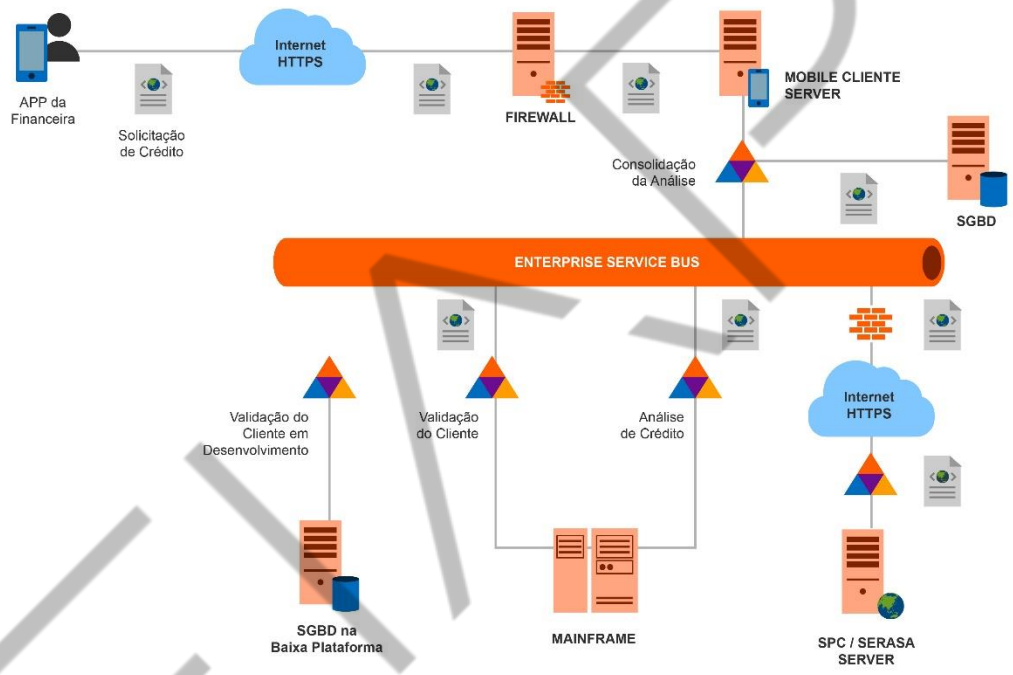

Figura 2.4 – Representação em alto nível do cenário de Solicitação de Crédito com a Validação do Cliente em Desenvolvimento na Baixa Plataforma
Fonte: Elaborado pelo autor (2017)

Quando o novo serviço estiver operacional, o Barramento deve ser configurado para chamar a validação que roda na plataforma baixa do banco. Dessa forma, nenhuma manutenção no App ou impacto no processo de análise e aprovação de crédito ocorrerá, observe a figura “Representação em alto nível do cenário de Solicitação de Crédito com a Validação do Cliente em Produção na Baixa Plataforma“ abaixo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

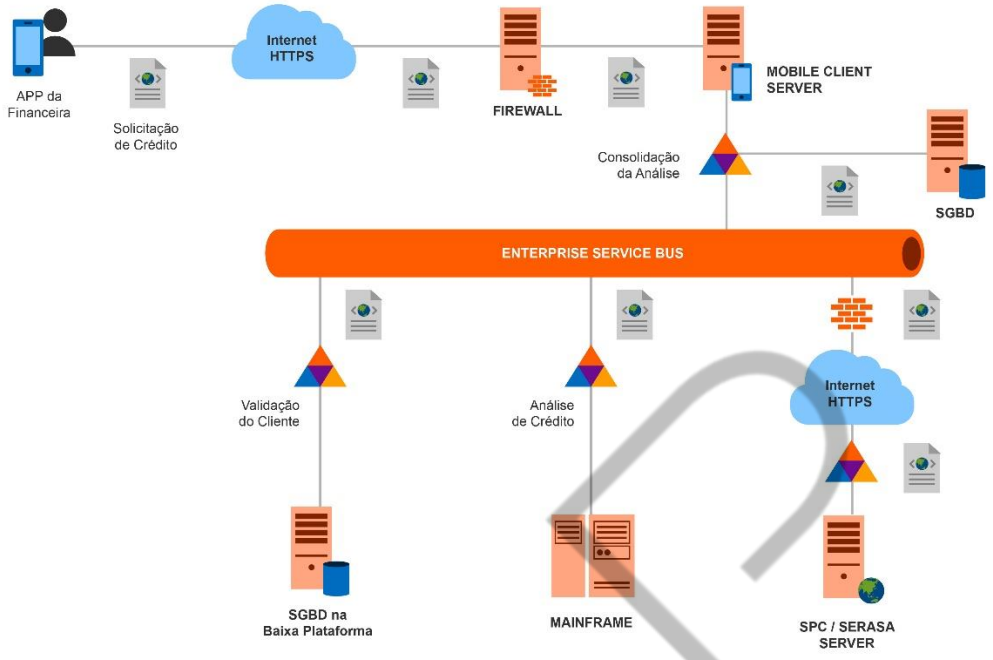

Figura 2.5 – Representação em alto nível do cenário de Solicitação de Crédito com a Validação do Cliente em Produção na Baixa Plataforma
Fonte: Elaborado pelo autor (2017)

Imagine comigo um outro cenário: o banco em questão adquiriu um banco XPTO e a migração da carteira de clientes ainda não ocorreu, mas o superintendente da financeira do banco comprador está ávido por novos contratos de financiamento. Nada incomum, concorda? Novos serviços acessando os dados dos clientes não migrados podem ser desenvolvidos pela financeira e plugados ao ESB do banco comprador, para rápida parametrização e nova orquestração alinhada ao processo de análise e aprovação de crédito do banco comprador.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

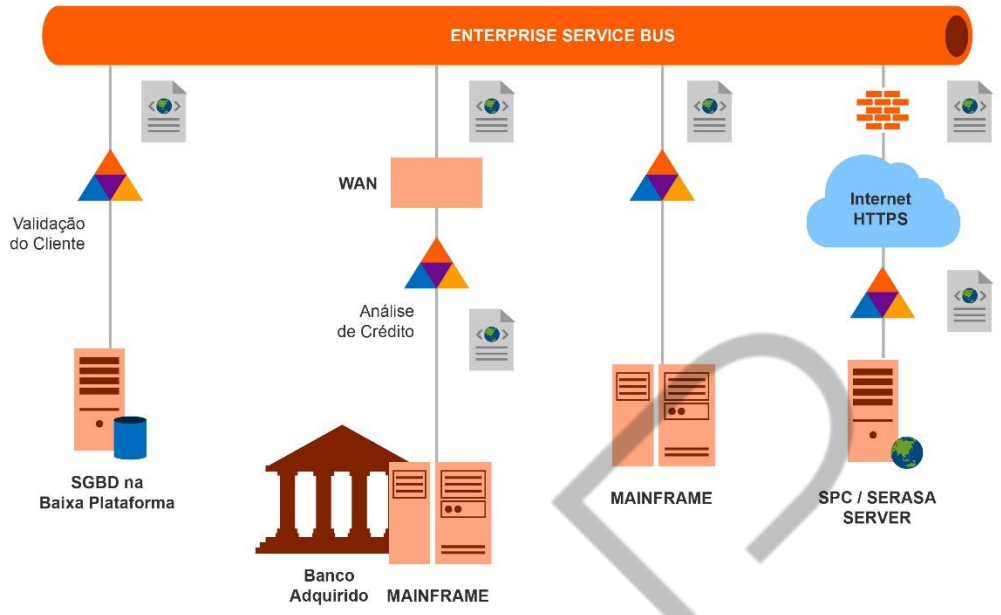

Figura 2.6 – Representação em alto nível do cenário de obtenção de dados de um cliente não migrado
Fonte: Elaborado pelo autor (2017)

É evidente que esforços e ajustes serão necessários. Desafios de cultura, infraestrutura e segurança da informação deverão ser superados, mas conceba como seria um processo de incorporação sem o acoplamento fraco gerado pela intermediação do Barramento. Pior, não?

O acoplamento fraco é importante para a flexibilidade tanto quanto para a escalabilidade. Sistemas distribuídos que suportam importantes processos de negócios estão sempre desafiando limites.

Na década de 1970, para consultar o saldo da sua conta bancária, você teria de ir a um caixa de uma agência ou utilizar um dos poucos ATMs disponíveis. Somente em 2013 o *Internet Banking* (IB) e o POS (*Point of Sale*), juntos, superaram esses canais mais tradicionais e pasme, nesse ano, o *Mobile Banking* (MB) ainda estava engatinhando e representava apenas 2,3% das transações bancárias no Brasil.

Poucos anos depois, em 2016, a Febraban informou que o *Mobile Banking* se tornou o canal mais utilizado pelos brasileiros para operações bancárias, passando a representar 34% das transações financeiras no país.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Pense comigo: os canais tradicionais levaram mais de quatro décadas para serem superados por novas tecnologias como *Internet Banking* e o POS. O *Mobile Banking* precisou de menos da metade de uma década para superar o IB junto ao POS.

Grande parte desse salto foi causado pela mudança de hábito do cliente que percebeu a facilidade de consultar o saldo pelo smartphone.

Para você, escalabilidade foi importante nesse contexto? Flexibilidade para atender os novos canais e a veloz mudança de comportamento dos clientes foi chave para TI ou não?

2.2 *Web Services*

2.2.1 Integrando aplicações corporativas, além das redes locais

Quando comentamos sobre as necessidades de negócios das grandes corporações e como SOA ajuda a TI a atendê-las, o conceito de serviços surgiu em todos os pontos. Expor uma funcionalidade de negócio para ser consumida por um sistema cliente fez parte de todas as soluções para os cenários de negócio que imaginamos.

Os *Web Services* são formas de expormos tais funcionalidades como serviço. O W3C (*World Wide Web Consortium*) e a OASIS (*Organization for the Advancement of Structured Informativo Standards*) são as organizações responsáveis por padronizar os *Web Services*. Empresas como IBM, Microsoft e HP, entre outras participantes do consórcio, apoiaram a criação dos padrões que determinam a tecnologia.

Segundo o W3C, a definição de *Web Services* é:

> *"A Web Service is a software system designed to support interoperable machine-to-machine interaction over a network." (W3C Working Group Note, 11 February 2004)*

Ou seja, um sistema de software projetado para suportar a interoperabilidade entre máquinas em uma rede. Em linhas gerais, podemos dizer que *Web Services* são aplicações desenvolvidas para integrar sistemas que se comunicam com fraco

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

acoplamento, por meio de mensagens que trafegam pela Internet ou de uma Intranet, partindo e retornando de ambientes heterogêneos.

Para essa tecnologia, pouco importa se o *Web Service* foi desenvolvido em uma linguagem de programação e é executado em um sistema operacional ou plataforma de hardware completamente diferente do sistema cliente que consome o serviço. Para que a interação ocorra, basta que eles se comuniquem, utilizando mensagens enviadas sobre o protocolo HTTP, comumente com os dados serializados nos formatos XML ou JSON.

Vamos voltar ao cenário do cliente consultando o saldo bancário via *Mobile Banking*. No contexto acima, o App do banco é caracterizado como o sistema cliente, responsável por consumir o serviço do *Internet Banking* que retornará o saldo bancário. Novamente, abstraindo inúmeros detalhes de infraestrutura e segurança, vejamos como será a solução utilizando *Web Services*.

O cliente do banco deverá selecionar os números, previamente cadastrados da agência e da conta, para na sequência digitar a respectiva senha em um formulário do App e disparar a solicitação de consulta.

O aplicativo, então, será responsável por criar uma mensagem serializada no formato XML ou JSON que terá como parte do conteúdo, os dados fornecidos pelo cliente do banco. Mensagem criada, o App deverá enviá-la pela Internet sobre o protocolo HTTPS para o *Web Service* responsável pela consulta de saldo do *Internet Banking*.

Após a autenticação realizada por um componente corporativo, o *Web Service* do IB será responsável por criar uma mensagem de retorno, igualmente serializada no formato XML ou JSON, que terá como parte do conteúdo o saldo da conta. O *Web Service* retornará a mensagem pela Internet ao sistema cliente (App), também utilizando o protocolo HTTPS. Ao receber a comunicação, o App deverá desserializar a mensagem em um objeto e obter o saldo por meio de um método público do objeto instanciado, para assim, apresentar o valor na tela de consulta do App.

Claro que o processo foi descrito de forma muito sucinta, não imagine que a codificação da consulta ao saldo da conta corrente estará dentro do código do Web Service. Com certeza, o *Web Service* utilizará outros componentes de software que são responsáveis pela consulta de forma corporativa em uma base no mainframe do

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

banco. Esses componentes retornarão o saldo para que o *Web Service* apenas crie a mensagem e atenda ao cliente que consumiu o serviço.

Esses mesmos componentes que realizam a consulta de saldo para o *Web Service*, atendem a qualquer outro canal que o cliente do banco esteja usando. Trocando em miúdos, se o cliente consultar o saldo da conta corrente pelo IB ou por um ATM ou por um caixa do banco, os mesmos componentes de software deverão ser chamados para realizar a consulta e devolver o resultado ao canal chamador. Lembra do acoplamento fraco que tanto comentamos?

Nesse exemplo, utilizamos a tecnologia de *Web Services* para atender ao canal Mobile e empregamos os conceitos da Arquitetura Orientada a Serviços para atender a esse canal ou a qualquer outro disponibilizado ao cliente que deseja realizar uma consulta de saldo.

Conceitualmente, pelos exemplos, percebemos que um *Web Service* é um software que possibilita ao provedor de serviços atender a consumidores trocando mensagens pela rede. Os sistemas que se comunicaram pouco sabem sobre as capacidades tecnológicas envolvidas. O App desconhece se o *Web Service* foi codificado em Java EE, .NET ou PHP e o *Web Service* não se preocupou se o App foi desenvolvido em Java Android, Swift ou alguma linguagem híbrida.

Isso acontece, pois tanto na requisição quanto na resposta, falamos sobre a necessidade de serializar as mensagens em formatos específicos XML ou JSON. Vamos pensar na parte técnica?

Em Java JEE, empregam-se dois termos para distinguir os padrões mais utilizados para codificação de *Web Services*: "*Big" Web Services* e *RESTful Web Services*. Nos nossos exemplos, se a arquitetura da solução definida empregasse XML/SOAP, teríamos codificado um *Big Web Service*, caso JSON, nossa solução teria sido desenvolvida em REST.

O JAX-WS (*Java API for XML Web Service*) é uma API Java EE, implementada em um projeto de código aberto, parte do projeto *Glassfish*, que provê funcionalidades para desenvolvermos os *"Big" Web Services*. Padrão que utiliza mensagens XML com o protocolo SOAP (*Simple Object Access Protocol*).

Para o RESTful (*Representational State Transfer*) o Java EE, possui o JAX-RS, também uma API, mas que provê funcionalidades para desenvolvermos *Web Services*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

no padrão REST. A implementação de referência para esta API é um projeto chamado *Jersey*. O *Jersey* implementa as definições da especificação JAX-RS, facilitando em muito a codificação de *Web Services* REST em Java.

Apenas como histórico, na criação dos padrões para *Web Services*, as empresas participantes do W3C adotaram o protocolo SOAP baseado em XML. O REST surgiu em 2000, na UC Irvine em uma tese de doutorado (PHD) e passou a compor a definição de *Web Services* do W3C em meados de 2004.

Ao longo deste capítulo, vamos apresentar mais detalhes de cada tecnologia envolvida e, claro, aprenderemos a criar soluções nos dois padrões, com maior profundidade em REST, devido a sua larga adoção pelo mercado. Você está preparado? Vamos lá!

2.2.2 Desenvolvendo *"Big" Web Services* com Apache CXF

O Apache CXF ™ é um framework para desenvolvimento de *Web Services* com código aberto. Foi criado por profissionais da Apache Software Foundation a partir da combinação de dois outros projetos abertos, o Celtix e o XFire. Por isso, a composição no nome CXF.

O CXF nos ajuda a construir e desenvolver *Web Services* usando as APIs de programação JAX-WS e o JAX-RS. Os serviços desenvolvidos com CXF podem utilizar vários protocolos, como SOAP, XML / HTTP, RESTful HTTP ou CORBA e trafegar em uma variedade de transportes, tais como HTTP, JMS ou JBI. Em nossos exemplos para *"Big" Web Services* utilizaremos JAX-WS, SOAP, XML / HTTP.

Antes de começarmos a desenvolver, considero importante que você compreenda como utilizaremos essa sopa de letras em nosso primeiro exemplo e os motivos pelos quais escolhemos o CXF como framework.

Ingredientes da sopa:

- JAX-WS será a API JEE que utilizaremos;
- Apache CXF será o framework de desenvolvimento;
- SOAP (*Simple Object Access Protocol*) – Protocolo que utilizaremos como definição da arquitetura das mensagens;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- XML (*eXtensible Markup Language*) – Para formatação das mensagens que conterão os dados enviados e recebidos;
- WSDL (*Web Services Description Language*) – Um arquivo XML que funciona como um contrato para descrever a interface de um Web Service;
- O WSDL será responsável por explicar como utilizar o *Web Service* que desenvolveremos para qualquer cliente em qualquer plataforma. O arquivo WSDL descreverá os métodos, argumentos, tipos de dados e a localização do nosso *Web Service*. Como o WSDL é formatado em XML, ele se torna um documento facilmente interpretável por humanos, tanto quanto por softwares. Essa característica possibilita aos sistemas clientes chamar e utilizar o nosso serviço dinamicamente;
- O WSDL também será utilizado para criarmos uma aplicação cliente com o gerador de código do Eclipse e o framework CXF;
- O Eclipse será a IDE e a ferramenta de monitoramento HTTP.
- O Maven gerenciará as dependências do nosso primeiro projeto;
- Para consumir o serviço, utilizaremos a ferramenta SoapUI, o próprio Eclipse e a aplicação cliente que codificaremos.

Por que utilizar CXF?

- O CXF possui alta performance e é extensível;
- Implementa as APIs JAX-WS de forma intuitiva e simples para nós usarmos;
- O CXF gera arquivos WSDL a partir de classes Java e gera classes Java a partir de WSDL facilmente;
- Suporta os padrões WS-* e JSR;
- Suporta JavaScript e ECMAScript 4 XML (E4X);
- Trabalha com múltiplos *Transports* (HTTP, Servlet, JMS, In-VM e outros), *Protocol Bindings* (SOAP, REST/HTTP e XML puro), *Data Bindings* (JAXB 2.x, Aegis, Apache XMLBeans, Service Data Objects (SDO) e JiBX) e Formatos (XML Textual, JSON e FastInfoset);

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Possui *deploy* flexível em vários servidores de aplicação (Apache Geronimo, JOnAS, Redhat JBoss, OC4J, Oracle WebLogic e IBM WebSphere) ou em containers JBI (Java Business Integration) ou em containers leves (Jetty, Tomcat ou Spring-based containers) ou até *standalone* (sem servidor ou container);
- Atualmente, está em forte desenvolvimento pelo time Apache. Em nossos exemplos utilizaremos o CXF 3.0.14, lançado em 29/07/2017.

2.2.2.1 Exemplo 1 - *Web Service standalone* com JAX-WS e CXF

Neste exemplo, criaremos um *Web Service* JAX-WS com CXF que será executado sem um servidor ou container.

Objetivos:

- Apresentar como criar um *Standalone Web Service*:
  - Criar um *Dynamic Web Project* e convertê-lo para Maven Project;
  - Configurar o arquivo pom para que o Maven gerencie as dependências;
  - Codificar a entidade e a implementação do serviço;
  - Codificar a interface do *Web Service*;
  - Publicar e executar o serviço com o WSDL disponível.
- Demonstrar como testar o serviço com a ferramenta SOAP UI:
  - Criar um projeto SoapUI;
  - Configurar o projeto para consumir o serviço através do WSDL gerado;
  - Apresentar o SOAP;
  - Testar o *Web Service* demonstrando a interoperabilidade.
- Desenvolver um cliente Java por meio de um gerador de código:
  - Instalar o CXF no Eclipse;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

  - Criar um Dynamic Web Project para ser o cliente;
  - Gerar as classes clientes através do Eclipse com CXF;
  - Consumir o serviço.
- Utilizar um monitor SOAP no Eclipse para monitorar a comunicação entre os clientes e o *Web Service*:
  - Configurar o monitor TCP/IP no Eclipse;
  - Monitorar e entender os *Headers* HTTP (*Request* e *Response*).

Pré-requisitos:

- Eclipse IDE for JEE Developers com Maven;
- Java JDK 1.8;
- SoapUI OpenSource (https://www.soapui.org/downloads/soapui.html).

Criando um Dynamic Web Project no Eclipse em *File* > *New* > *Dynamic Web Project*:

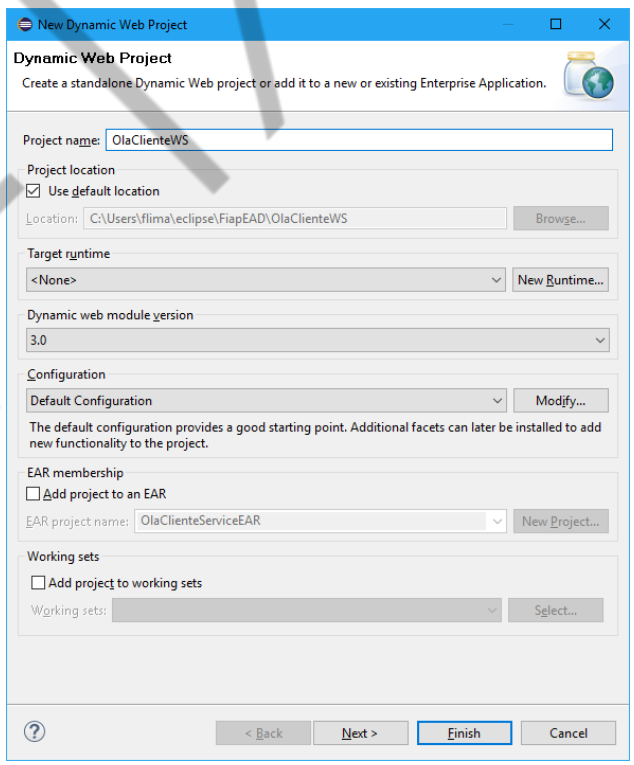

Figura 2.7 – Tela de criação do projeto OlaClienteWS no Eclipse
Fonte: Elaborado pelo autor (2017)

Convertendo o projeto para Maven Project:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

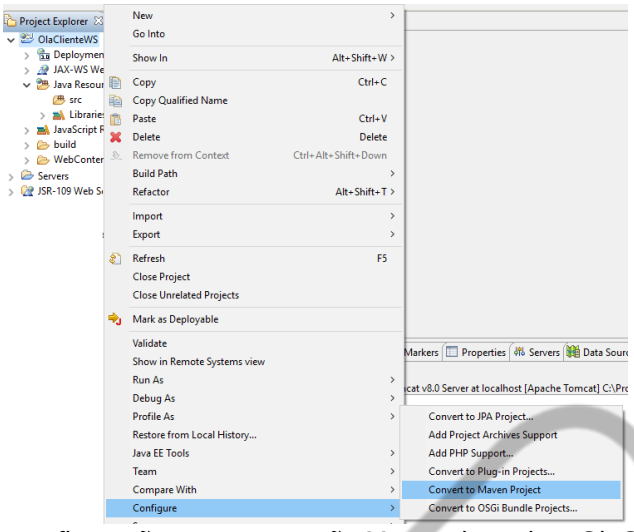

Figura 2.8 – Menu de configuração para conversão Maven do projeto OlaClienteWS no Eclipse
Fonte: Elaborado pelo autor (2017)

Nomeando o projeto Maven:

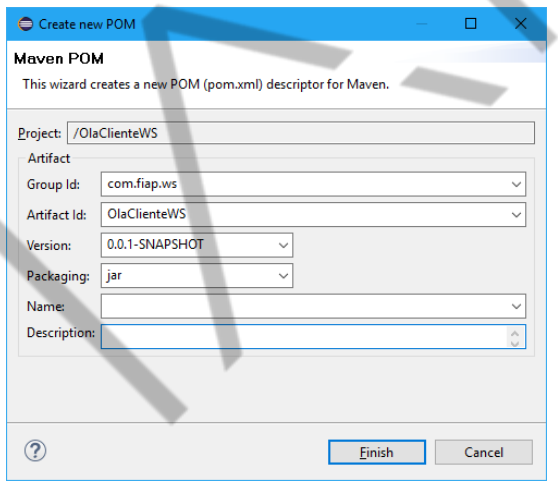

Figura 2.9 – Tela de criação do arquivo pom do projeto OlaClienteWS no Eclipse
Fonte: Elaborado pelo autor (2017)

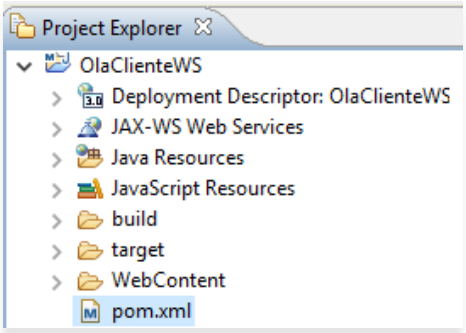

Figura 2.10 – Resultado da criação do arquivo pom do projeto OlaClienteWS no Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Configurando o arquivo pom para que o Maven gerencie as dependências CXF.
As dependências estão disponíveis em:

https://mvnrepository.com/artifact/org.apache.cxf/cxf-rt-frontend-jaxws/3.1.12

https://mvnrepository.com/artifact/org.apache.cxf/cxf-rt-transports-http/3.1.12

https://mvnrepository.com/artifact/org.apache.cxf/cxf-rt-transports-http-jetty/3.1.12

Dê um duplo click no arquivo pom.xml e, na aba pom.xml do eclipse, inclua as dependências rt-frontend-jaxws, rt-transports-http e transports-http-jetty:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
      <modelVersion>4.0.0</modelVersion>
      <groupId>com.fiap.ws</groupId>
      <artifactId>OlaClienteWS</artifactId>
      <version>0.0.1-SNAPSHOT</version>
      <dependencies>
            <dependency>
                  <groupId>org.apache.cxf</groupId>
                        <artifactId>
                              cxf-rt-frontend-jaxws
                        </artifactId>
                  <version>3.1.12</version>
            </dependency>
            <dependency>
                  <groupId>org.apache.cxf</groupId>
                        <artifactId>
                              cxf-rt-transports-http
                        </artifactId>
                  <version>3.1.12</version>
            </dependency>
            <dependency>
                  <groupId>org.apache.cxf</groupId>
                  <artifactId>
                        cxf-rt-transports-http-jetty
                  </artifactId>
                  <version>3.1.12</version>
            </dependency>
      </dependencies>
      <build>
            <sourceDirectory>src</sourceDirectory>
            <plugins>
                  <plugin>
                        <artifactId>
                              maven-compiler-plugin
                        </artifactId>
                        <version>3.3</version>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                        <configuration>
                              <source>1.8</source>
                              <target>1.8</target>
                        </configuration>
                  </plugin>
            </plugins>
      </build>
</project>
```

Código-fonte 2.1 – *Documento pom*
Fonte: Elaborado pelo autor (2017).

Codificando a entidade e a implementação do serviço:

- Crie a classe Cliente no pacote com.fiap.bank.entity

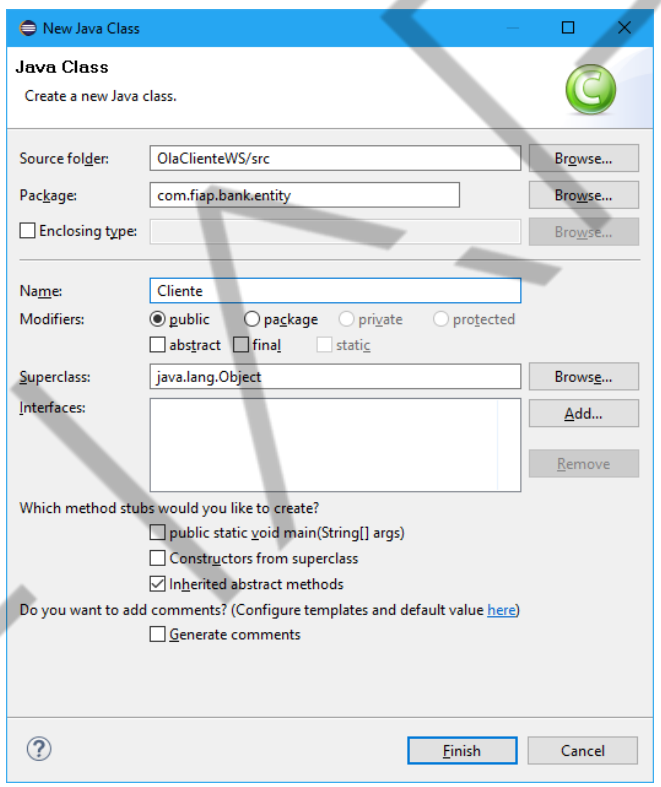

Figura 2.11 – Tela de criação da classe Cliente no Eclipse
Fonte: Elaborado pelo autor (2017)

Codifique uma entidade cliente bem simples. Gere os métodos com o atalho CTRL+3 digitando ggas, opções:

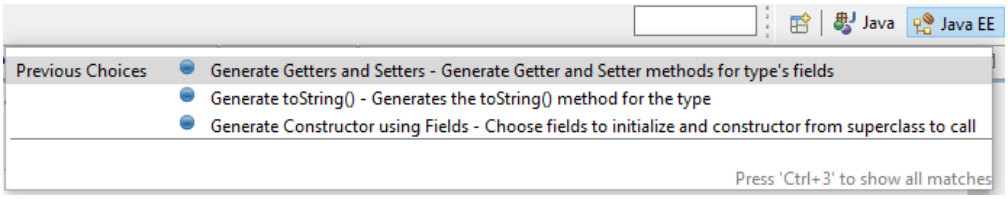

Figura 2.12 – Menu de atalhos no Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```java
package com.fiap.bank.entity;

public class Cliente {

    private int idCliente;
    private String nome, sobrenome, email;

    public Cliente(int idCliente, String nome, String
sobrenome, String email) {
        super();
        this.idCliente = idCliente;
        this.nome = nome;
        this.sobrenome = sobrenome;
        this.email = email;
    }

    public Cliente() {}

    public int getIdCliente() {
        return idCliente;
    }
    public void setIdCliente(int idCliente) {
        this.idCliente = idCliente;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getSobrenome() {
        return sobrenome;
    }
    public void setSobrenome(String sobrenome) {
        this.sobrenome = sobrenome;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }

    @Override
    public String toString() {
        return "Cliente [idCliente="
                                + idCliente
                                + ", nome="
                                + nome
                                + ", sobrenome="
                                + sobrenome
                                + ", email="
                                + email
                                + "]";
    }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        }
```

Código-fonte 2.2 – classe Cliente
Fonte: Elaborado pelo autor (2017)

- Crie uma interface para o nosso *Web Service* no pacote com.fiap.bank.ws

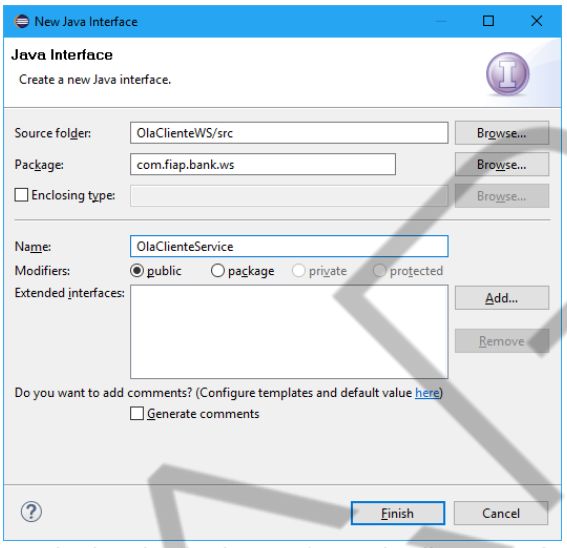

Figura 2.13 – Tela de criação da Interface OlaClienteService no Eclipse
Fonte: Elaborado pelo autor (2017)

```
package com.fiap.bank.ws;

import javax.jws.WebParam;
import javax.jws.WebService;
import com.fiap.bank.entity.Cliente;

@WebService
public interface OlaClienteService {
     String digaOla(@WebParam(name= "cliente")
                                        Cliente cliente);
}
```

Código-fonte 2.3 – interface OlaCliente
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Agora, ainda no pacote com.fiap.bank.ws, crie a implementação do nosso *Web Service*:

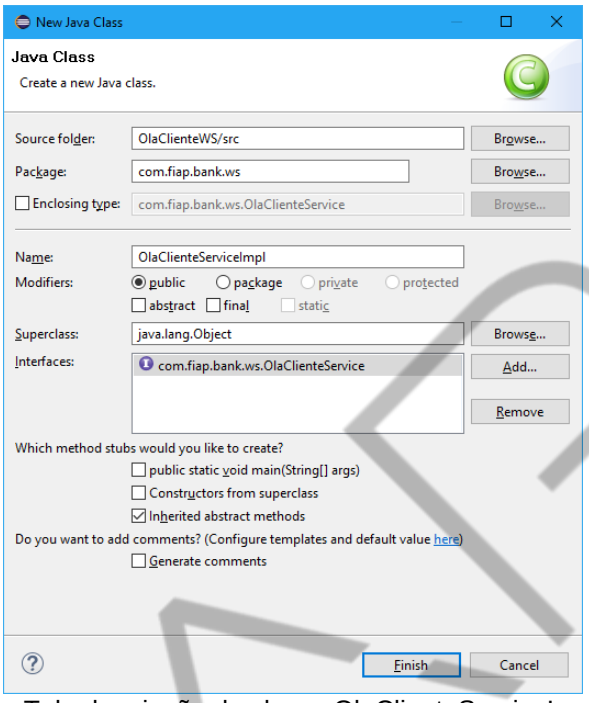

Figura 2.14 – Tela de criação da classe OlaClienteServiceImpl no Eclipse
Fonte: Elaborado pelo autor (2017)

```
package com.fiap.bank.ws;

import com.fiap.bank.entity.Cliente;
import javax.jws.WebService;

@WebService
public class OlaClienteServiceImpl
                    implements OlaClienteService {

     @Override
     public String digaOla(Cliente cliente) {
          return "Olá " + cliente.toString();
     }

}
```

Código-fonte 2.4 – classe OlaClienteServiceImpl
Fonte: Elaborado pelo autor (2017)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Publicando e executando o serviço com o WSDL disponível:

- Crie a classe *RunService* no pacote com.fiap.bank.server. Classe que será responsável por publicar e executar o nosso *Web Service*.

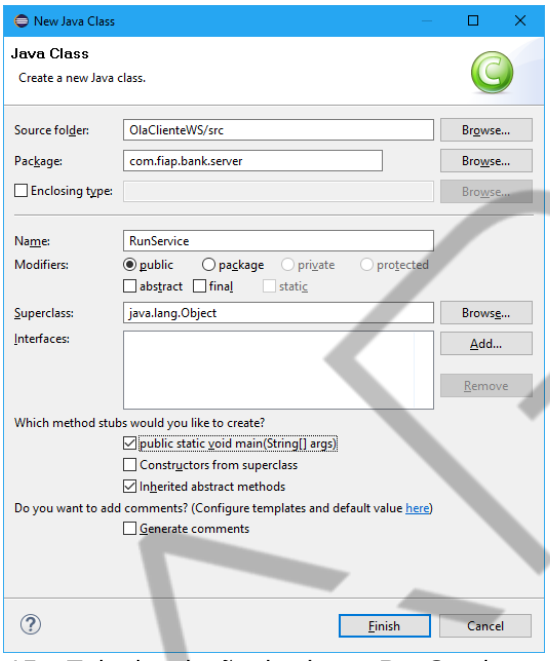

Figura 2.15 – Tela de criação da classe *RunService* no Eclipse
Fonte: Elaborado pelo autor (2017)

```
package com.fiap.bank.server;

import org.apache.cxf.jaxws.JaxWsServerFactoryBean;
import com.fiap.bank.ws.OlaClienteService;
import com.fiap.bank.ws.OlaClienteServiceImpl;

public class RunService {

	public static void main(String[] args) {

		try {
			OlaClienteService clienteWS =
				new OlaClienteServiceImpl();

			JaxWsServerFactoryBean factory =
				new JaxWsServerFactoryBean();
			factory.setServiceClass(OlaClienteService.class);
			factory.setAddress("http://localhost:8809/digaola");
			factory.setServiceBean(clienteWS);
			factory.create();
			System.out.println("Serviço operacional");
		} catch (Exception e) {
			e.printStackTrace();
		}
	}
}
```

Código-fonte 2.5 – classe OlaClienteServiceImpl
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O *JaxWsServerFactoryBean* é um *bean* CXF feito para ajudar a criar facilmente *Server endpoints* para o JAX-WS. Um *endpoint* é um endereço da Web (URL) que os clientes consumidores de um serviço podem acessar e utilizar as operações realizadas por esse serviço.

- Execute a classe *RunService* como uma Aplicação Java.

  Run As > *Java Application* ou (Alt + Shift + X, J)

Figura 2.16 – Console exibindo log de execução no Eclipse
Fonte: Elaborado pelo autor (2017)

- Acesse o WSDL gerado dinamicamente pelo CXF em http://localhost:8809/digaola?wsdl. Ele contém todas as informações necessárias para que um cliente consuma o nosso *Web Service*:

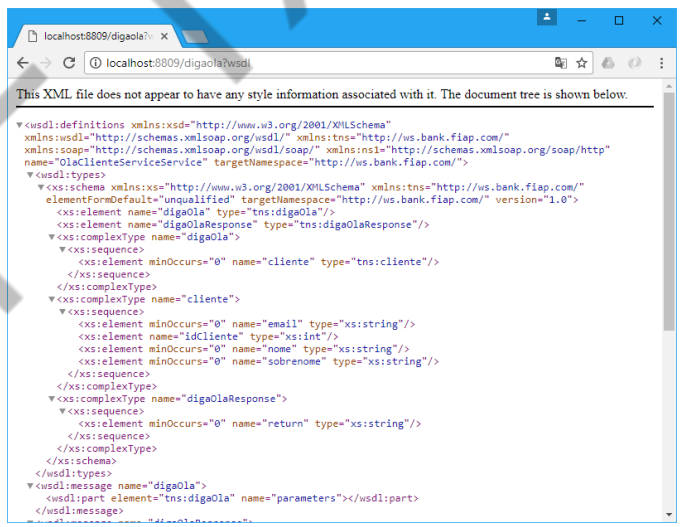

Figura 2.17 – Browser Chrome exibindo um wsdl local
Fonte: Elaborado pelo autor (2017)

Como testar o serviço com a ferramenta SOAP UI:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Crie um projeto SoapUI e o configure para consumir o serviço através do WSDL gerado;

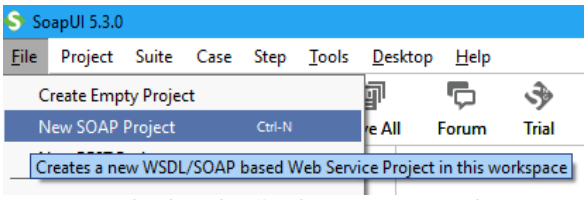

Figura 2.18 – Tela de criação de um novo projeto no SoapUI
Fonte: Elaborado pelo autor (2017)

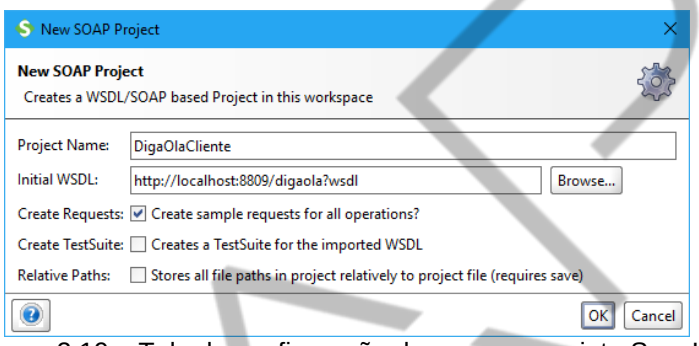

Figura 2.19 – Tela de configuração de um novo projeto SoapUI
Fonte: Elaborado pelo autor (2017)

- Dê um click no *Request* 1 do projeto e preencha os dados do cliente no corpo da mensagem envelopada pelo protocolo SOAP;

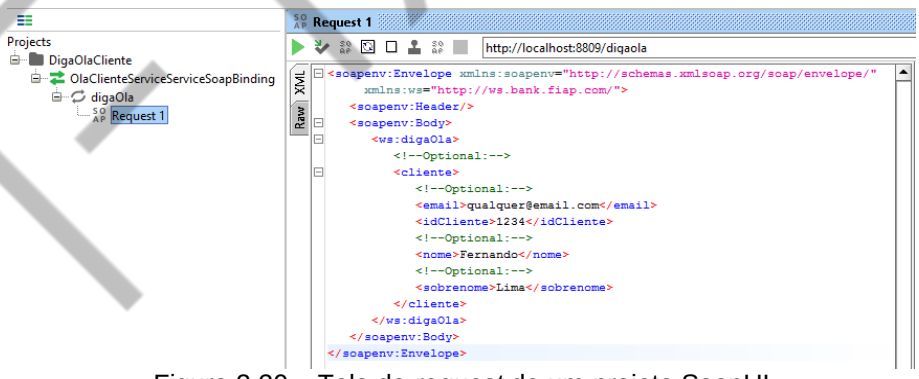

Figura 2.20 – Tela de *request* de um projeto SoapUI
Fonte: Elaborado pelo autor (2017)

- Execute o consumo do serviço e verifique o retorno;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

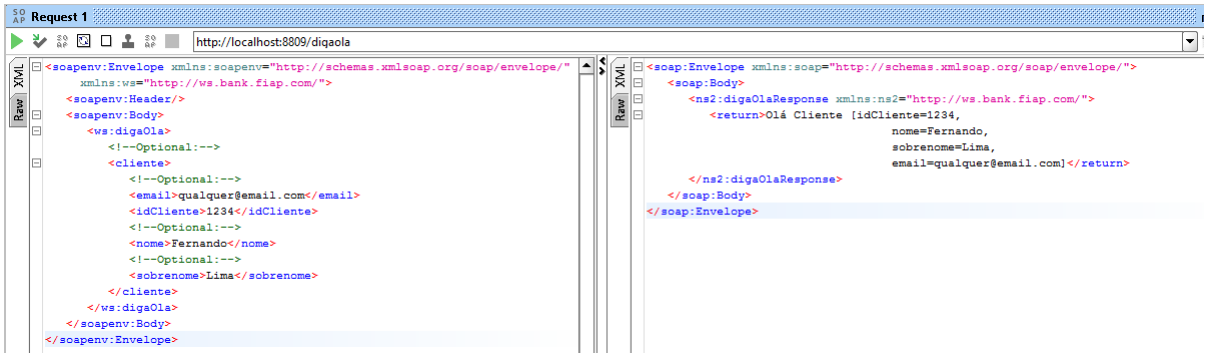

Figura 2.21 – Tela exibindo *request* / response de um projeto SoapUI
Fonte: Elaborado pelo autor (2017)

Reflita comigo, o SoapUI nada sabia sobre o nosso serviço até utilizar o WSDL. Concorda?

O nosso *Web Service* conhecia o SoapUI? Para o serviço foi importante em qual linguagem de programação o SoapUI foi codificado ou em qual sistema operacional ele foi executado no momento do consumo?

Ficou mais clara a interoperabilidade sobre a qual falamos?

O arquivo WSDL, gerado dinamicamente pelo CXF, foi capaz de explicar remotamente ao cliente SoapUI como consumir o nosso serviço utilizando o método digaOla, com o XML Element <ws:digaOla>... </ws:digaOla> e como enviar o parâmetro que o método esperava com o Element <cliente> ... </cliente>, uma representação em XML do objeto Cliente.

Veja que a mensagem de *request* também foi gerada dinamicamente, só que em SOAP e a partir do WSDL lido pelo SoapUI. O SOAP foi utilizado pelo SoapUI para estruturar a mensagem de requisição e pelo nosso *Web Service* para estruturar a mensagem de resposta. Ambas mensagens trafegaram entre o cliente SoapUI e o endpoint http://localhost:8809/digaola por meio do protocolo de rede HTTP.

Software conversando com software. Nesse exemplo, a intervenção humana foi limitada a apontar o WSDL na rede e preencher os dados do cliente. Agora, leia o WSDL no seu browser e veja que ele é acessível tanto para máquinas quanto para humanos. Legal, não?

Uma dúvida pode estar pairando em sua cabeça, mas como podemos descobrir *"Big" Web Services* na Internet? Falaremos disso mais à frente, quando

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

discorrermos sobre UDDI (*Universal Description, Discovery and Integration*). Agora, vamos praticar mais um pouco.

Desenvolvendo um cliente Java, por meio do gerador de código Eclipse / CXF:

- Vamos instalar o CXF no Eclipse. Precisamos dele para criar um cliente Java pelo gerador de código da IDE. Baixe o arquivo apache-cxf-3.1.12.zip da URL http://cxf.apache.org/download.html.

3.1.12

The 3.1.12 release is our latest release representing a significant amount of work on new features, enhancements, code cleanups, etc... For a complete list of new features, API changes, etc... please see the release notes and migration guide.

Description	File	MD5	SHA1	PGP
Source distribution	apache-cxf-3.1.12-src.tar.gz	apache-cxf-3.1.12-src.tar.gz.md5	apache-cxf-3.1.12-src.tar.gz.sha1	apache-cxf-3.1.12-src.tar.gz.asc
	apache-cxf-3.1.12-src.zip	apache-cxf-3.1.12-src.zip.md5	apache-cxf-3.1.12-src.zip.sha1	apache-cxf-3.1.12-src.zip.asc
Binary distribution	apache-cxf-3.1.12.tar.gz	apache-cxf-3.1.12.tar.gz.md5	apache-cxf-3.1.12.tar.gz.sha1	apache-cxf-3.1.12.tar.gz.asc
	apache-cxf-3.1.12.zip	apache-cxf-3.1.12.zip.md5	apache-cxf-3.1.12.zip.sha1	apache-cxf-3.1.12.zip.asc

Quadro 3.1 – Chrome exibindo a página de download do CXF 3.1.12
Fonte: Elaborado pelo autor (2017)

Descompacte os 75,2 MB do zip em um diretório que seja fácil de localizar. No Eclipse vá em *Window* > *Preferences* > *Web Services* > *CXF 2.x Preferences* e adicione o *runtime* do CXF 3.1.12, apontando o diretório raiz onde você descompactou o arquivo zip. Clique em *Finish*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

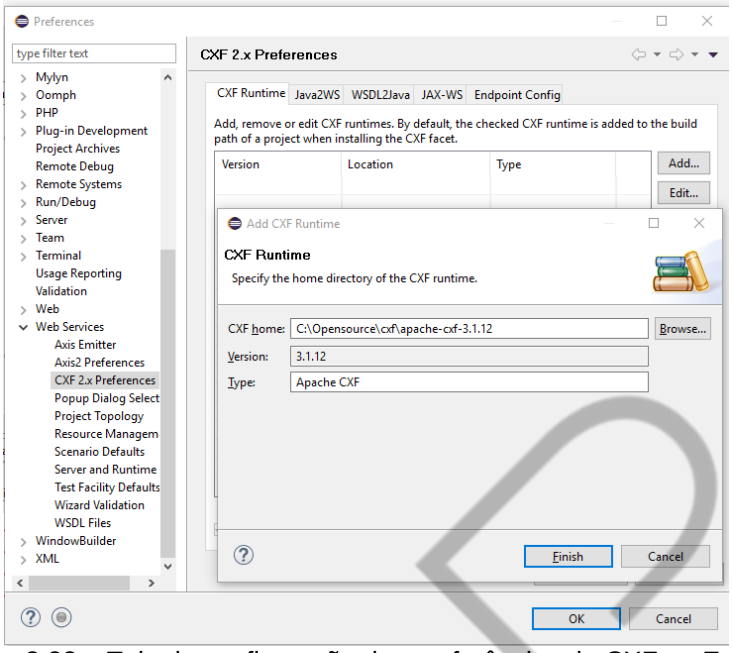

Figura 2.22 – Tela de configuração das preferências do CXF no Eclipse
Fonte: Elaborado pelo autor (2017)

Ao final, garanta que os dois *checkboxes* estão selecionados, conforme a imagem abaixo.

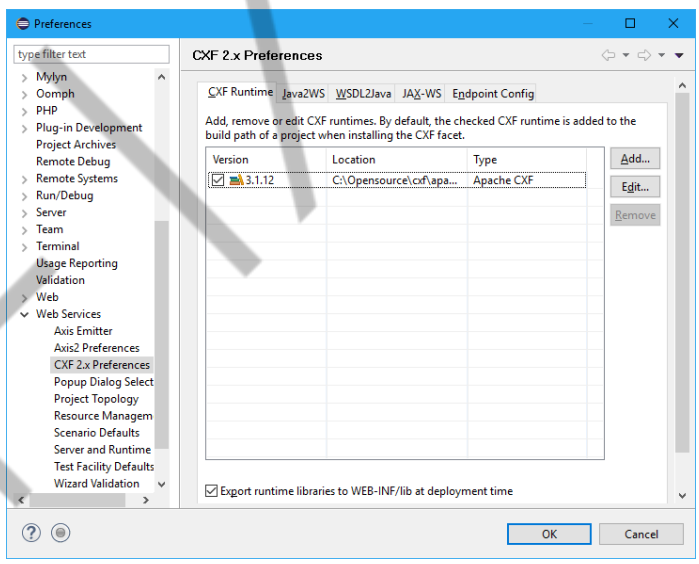

Figura 2.23 – Tela de configuração das preferências do CXF no Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Crie um *Dynamic Web Project* de nome OlaClienteConsumer para ser o nosso sistema cliente em Java.

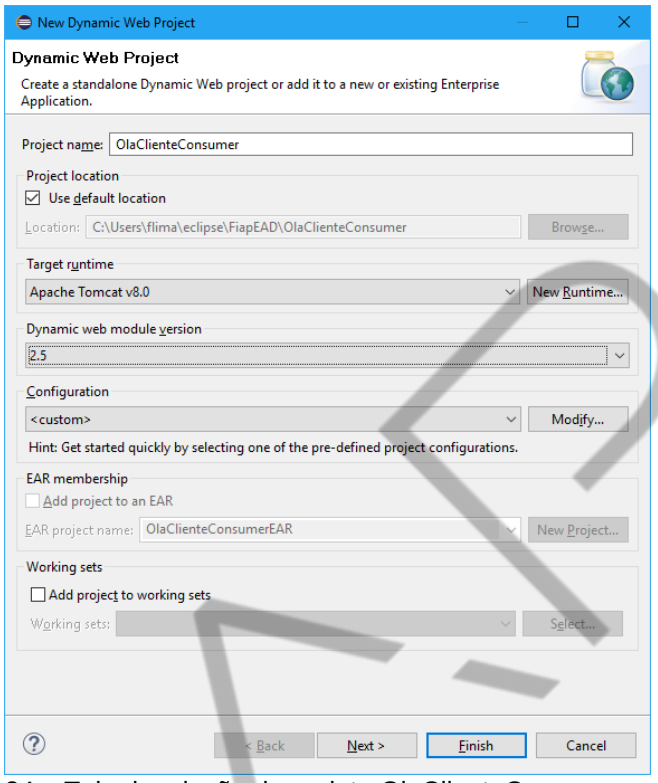

Figura 2.24 – Tela de criação do projeto OlaClienteConsumer no Eclipse
Fonte: Elaborado pelo autor (2017)

Clique com o botão direito em src e faça os passos a seguir:

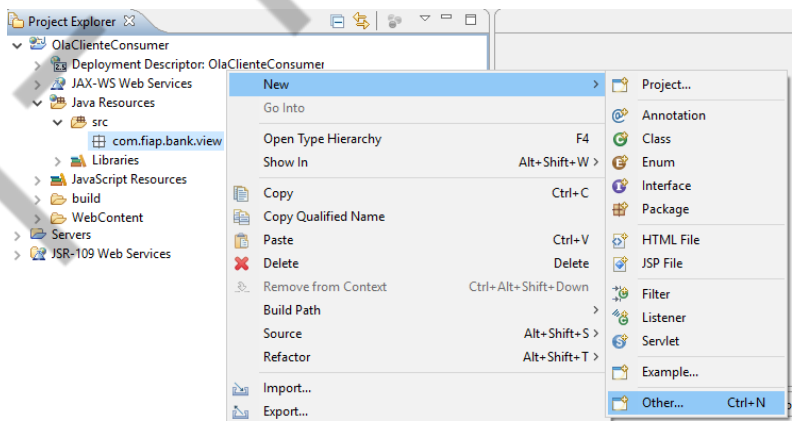

Figura 2.25 – Menu de navegação *New* > *Other* no Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

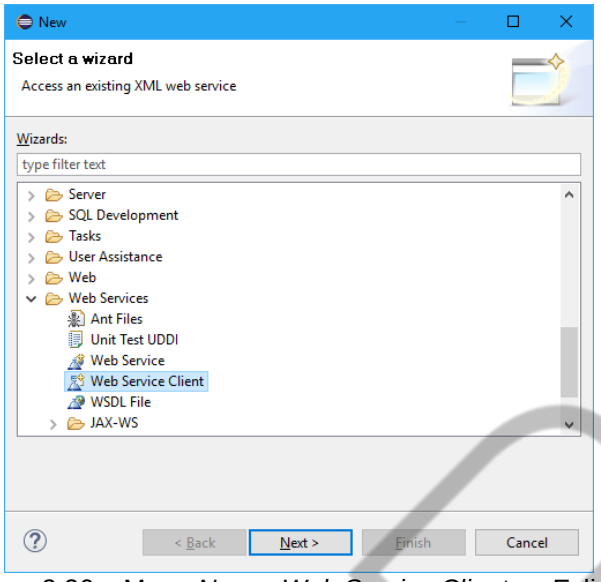

Figura 2.26 – Menu *New* > *Web Service Client* no Eclipse
Fonte: Elaborado pelo autor (2017)

Click em *Next* e digite a URL do WSDL do nosso serviço. Garanta que o nosso *Web Service* esteja em execução. Como vamos criar uma aplicação console, configure somente até *Develop client*, conforme imagem abaixo.

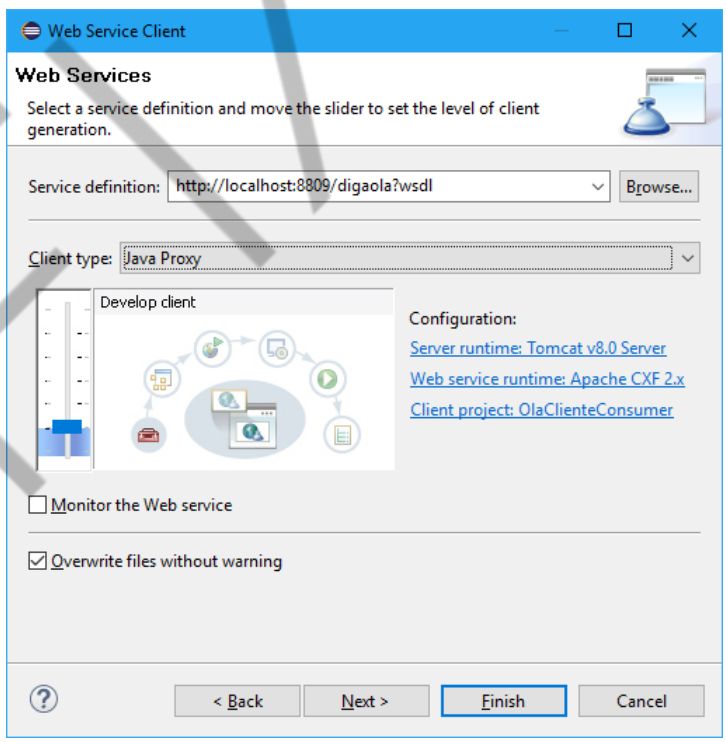

Figura 2.27 – Tela de geração de um *Web Service Client* no Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O gerador sugerirá o pacote abaixo. Click em *Finish*.

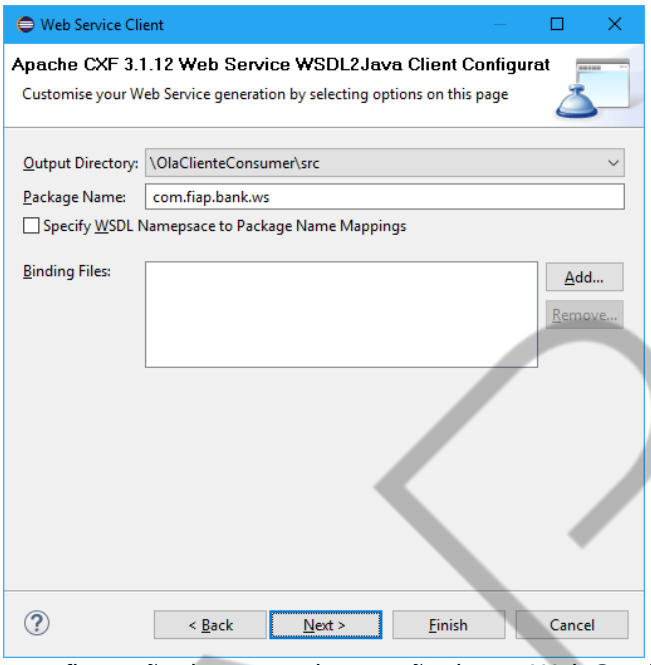

Figura 2.28 – Tela de configuração de pacote da geração de um *Web Service Client* no Eclipse
Fonte: Elaborado pelo autor (2017)

As seguintes classes serão geradas pelo Eclipse com o CXF a partir do WSDL, com tudo que precisamos para consumir o serviço. Então, vamos consumi-lo?

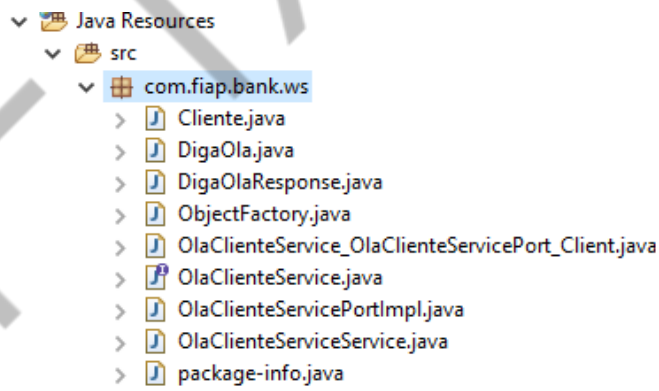

Figura 2.29 – Resultado da geração no Project Explore no Eclipse.
Fonte: Elaborado pelo autor (2017).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Crie uma classe de nome OlaClienteMain no pacote com.fiap.bank.view.

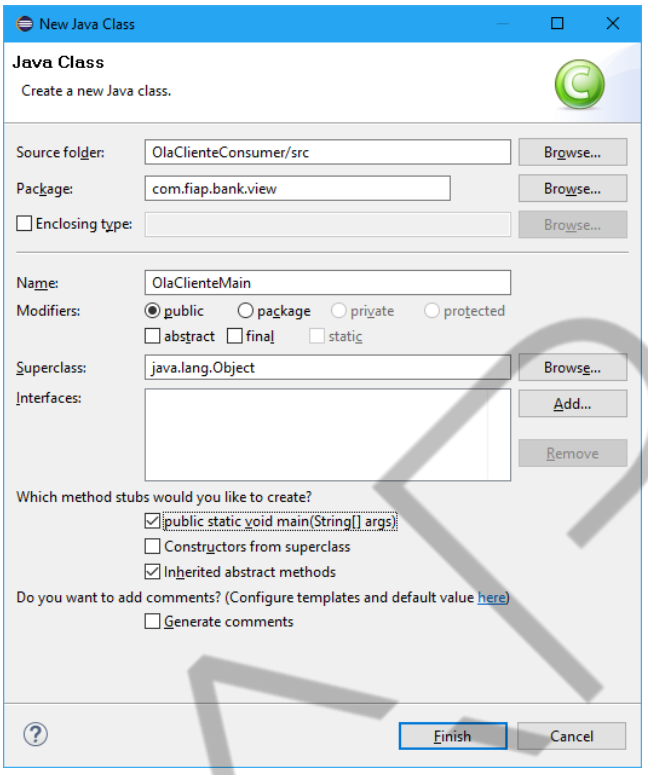

Figura 2.30 – Tela de criação da classe OlaClienteMain no Eclipse
Fonte: Elaborado pelo autor (2017)

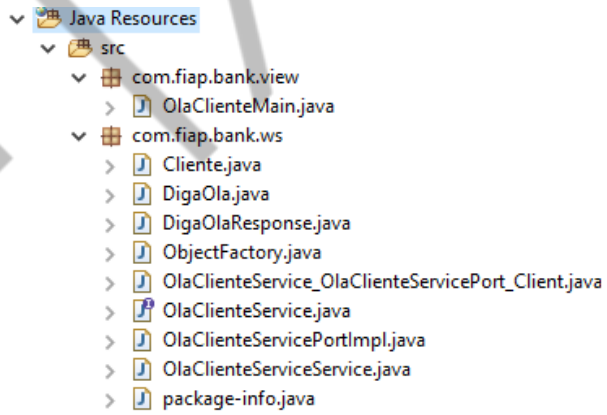

Figura 2.31 – Resultado da criação da classe OlaClienteMain no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Codifique a classe OlaClienteMain. Instancie um Cliente e atribua valores às propriedades da classe. Instancie um objeto OlaClienteServiceService e, por meio dele, consuma o método digaOla do nosso *Web Service* enviando como parâmetro o cliente cujo atributos setamos, conforme o código abaixo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
package com.fiap.bank.view;

import com.fiap.bank.ws.Cliente;
import com.fiap.bank.ws.OlaClienteServiceService;

public class OlaClienteMain {

      public static void main(String[] args) {

            Cliente cliente = new Cliente();

            cliente.setIdCliente(1);
            cliente.setEmail("qualquer@email.com");
            cliente.setNome("Fernando");
            cliente.setSobrenome("Lima");

            OlaClienteServiceService OlaCliente
                           = new OlaClienteServiceService();

            String retorno = OlaCliente
                                 .getOlaClienteServicePort()
                                 .digaOla(cliente);

            System.out.println(retorno);

      }

}
```

Código-fonte 2.6 – classe OlaClienteMain
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, vamos usar o monitor de TCP/IP do Eclipse para acompanharmos todas as mensagens trocadas entre o cliente que geramos com o Eclipse CXF e o nosso *Web Service*. No Eclipse navegue em *Window* > *Show View* > *Other* ...> TCP/IP Monitor e clique OK.

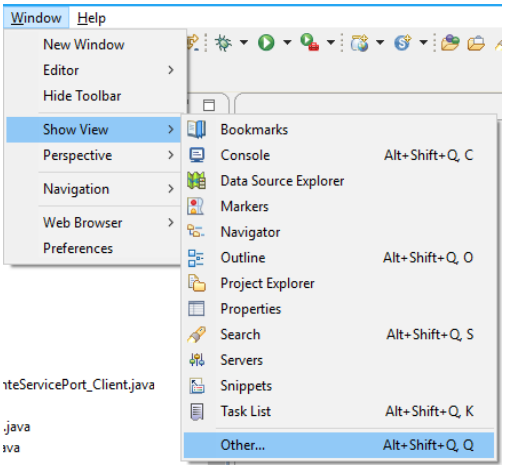

Figura 2.32 – Menu de navegação *Window* > *Show View* > *Other* no Eclipse
Fonte: Elaborado pelo autor (2017)

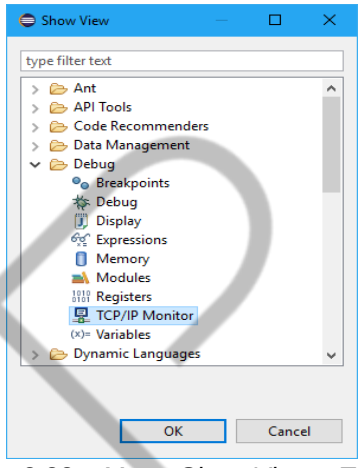

Figura 2.33 – Menu *Show View* > TCP/IP Monitor
Fonte: Elaborado pelo autor (2017)

Em *Windows* > *Preferences*, expanda *Run* / *Debug*, selecione em TCP / IP Monitor e clique em *Add*. Configure o monitor conforme a imagem "Tela de configuração *Add* TCP/IP Monitor no Eclipse".

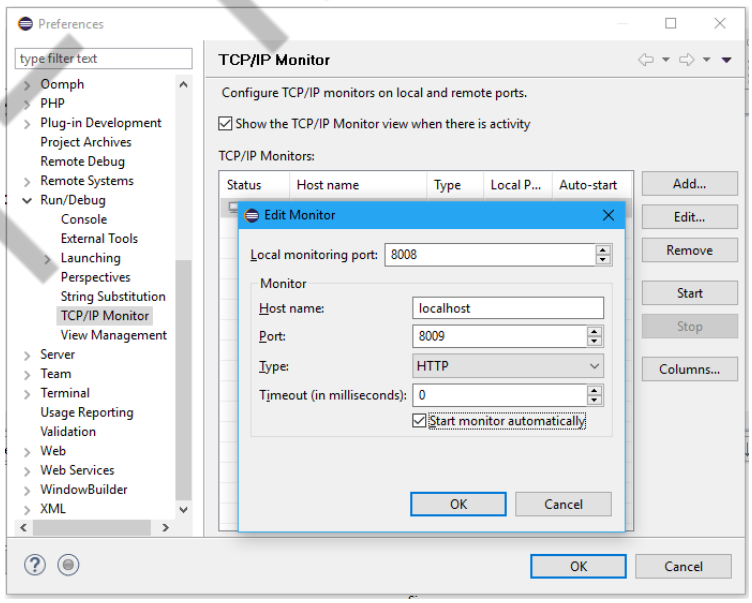

Figura 2.34 – Tela de configuração *Add* TCP/IP Monitor no Eclipse
Fonte: Elaborado pelo autor (2017)

Clique em *Start* e em OK.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

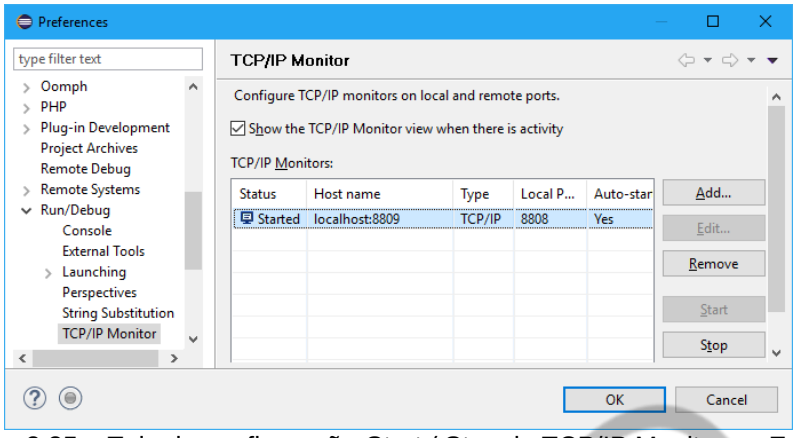

Figura 2.35 – Tela de configuração *Start / Stop* do TCP/IP Monitor no Eclipse
Fonte: Elaborado pelo autor (2017)

Com tal configuração, o monitor do Eclipse, quando em execução, receberá tudo que chegar à porta 8808 (*Local monitoring port*) para redirecionar ao nosso serviço na porta 8809 (*Port*). Nesse meio, ele capturará o que trafegar sobre o protocolo TCP/IP, ou seja, veremos na *view* do Eclipse o *header* HTTP e o SOAP, tanto da requisição cliente quanto da resposta do serviço.

Para tanto, precisamos refatorar a nossa classe OlaClienteMain, separando as requisições em dois métodos. Uma nova requisição que passará pelo monitor na porta 8808 e uma segunda, com a solução já implementada, que envia direto para o nosso serviço na porta 8809.

```
package com.fiap.bank.view;

import javax.xml.ws.BindingProvider;
import com.fiap.bank.ws.Cliente;
import com.fiap.bank.ws.OlaClienteService;
import com.fiap.bank.ws.OlaClienteServiceService;

public class OlaClienteMain {

      public static void main(String[] args) {

            Cliente cliente = new Cliente();
            cliente.setIdCliente(1);
            cliente.setEmail("qualquer@email.com");
            cliente.setNome("Fernando");
            cliente.setSobrenome("Lima");

            //String retorno = olaCliente(cliente);
            String retorno = olaClienteMonitor(cliente);

            System.out.println(retorno);
      }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //Método que passa pelo monitor na porta 8808
        private static String olaClienteMonitor(Cliente cliente) {
              OlaClienteServiceService olaCliente =
                                    new OlaClienteServiceService();

              OlaClienteService ocs =
                              (OlaClienteService)olaCliente
                                    .getOlaClienteServicePort();

              ((BindingProvider) ocs )
                              .getRequestContext()
                              .put(
        BindingProvider.ENDPOINT_ADDRESS_PROPERTY,

        "http://localhost:8808/digaola");

              return ocs.digaOla(cliente);
        }

        //Método com endpoit transparente, gerado pelo CXF
        private static String olaCliente(Cliente cliente) {
              OlaClienteServiceService OlaCliente
                              = new OlaClienteServiceService();

              return     OlaCliente
                                          .getOlaClienteServicePort()
                                          .digaOla(cliente);

        }
}
```

Código-fonte 2.7 – classe OlaClienteMain refatorada
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Com a chamada ao método olaClienteMonitor fora de comentário, execute o programa OlaClienteMain novamente. A saída no monitor será algo muito semelhante à imagem abaixo.

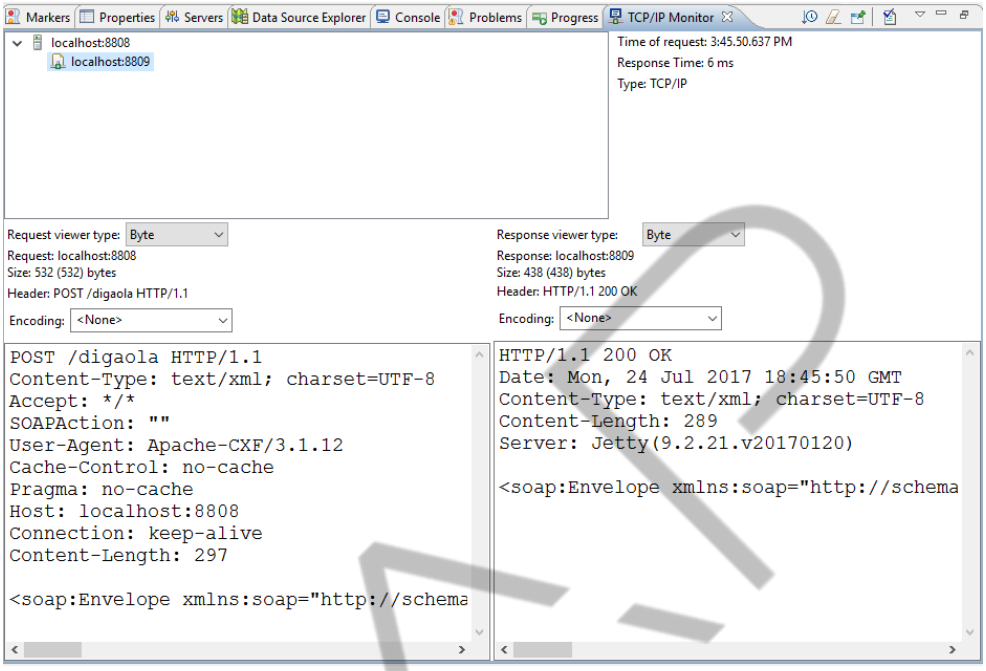

Figura 2.36 – View TCP/IP Monitor do Eclipse com o resultado do chamado ao *endpoint* digaola
Fonte: Elaborado pelo autor (2017)

Como a imagem mostra, ao executarmos o cliente novamente, o monitor capturou que foi feito um POST HTTP para o servidor localhost:8088, no *endpoint* localhost:8088/digaola, por um programa codificado em Apache-CXF/3.1.12 que enviou na requisição o tipo de conteúdo text/xml, contendo o seguinte envelope SOAP:

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
      <soap:Body>
            <ns2:digaOla xmlns:ns2="http://ws.bank.fiap.com/">
                  <cliente>
                        <email>qualquer@email.com</email>
                        <idCliente>1</idCliente>
                        <nome>Fernando</nome>
                        <sobrenome>Lima</sobrenome>
                  </cliente>
            </ns2:digaOla>
      </soap:Body>
</soap:Envelope>
```

Código-fonte 2.8 – classe OlaClienteMain
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como retorno, o monitor capturou o código HTTP 200 (*success status response code*). Código de sucesso do protocolo HTTP que informa que a requisição do sistema cliente foi bem-sucedida. O cabeçalho também demonstra que o servidor que respondeu a solicitação foi construído em Jetty. Bom momento para relembrar as dependências que incluímos no pom do nosso *Web Service*.

O *Web Service* retornou em txt/xml, contendo os dados do cliente enviados no POST, envelopados na seguinte mensagem SOAP:

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
     <soap:Body>
          <ns2:digaOlaResponse
xmlns:ns2="http://ws.bank.fiap.com/">
                <return>
                     Olá Cliente [idCliente=1, nome=Fernando,
sobrenome=Lima, email=qualquer@email.com]
                </return>
          </ns2:digaOlaResponse>
     </soap:Body>
</soap:Envelope>
```

Código-fonte 2.9 – classe OlaClienteMain
Fonte: Elaborado pelo autor (2017)

Esse exemplo, envolvendo um serviço e um cliente Java, demonstrou diversos pontos das tecnologias envolvidas, mais dois pontos relevantes ficaram de fora. Como a arquitetura criada pelo W3C tentou resolver a necessidade de descoberta de serviços na Internet e como executar o nosso de *"Big" Web Service* em um Servidor Web.

2.2.2.2 *Web Service Architecture* e UDDI

Vamos ao primeiro ponto, a *Universal Description, Discovery, and Integration* (UDDI) é um framework baseado em XML, independente de plataforma que foi concebido pelo W3C para registrar, descrever e descobrir serviços empresariais usando a Internet. A ideia do UDDI é possibilitar a integração de dois papéis que nós vimos bem: provedores de serviços (*Web Services*) e consumidores desses serviços (Aplicações Clientes) com um terceiro papel ainda não apresentado, *Brokers* de Serviços públicos ou privados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como isso funciona? As empresas que desejam fornecer serviços podem registrar informações sobre seus *Web Services* em *Brokers*, que hospedam uma implementação UDDI *Business Registry* (UBR). Assim, os consumidores podem procurar no UBR qualquer serviço desejado. Como se fosse uma página amarela de serviços na web.

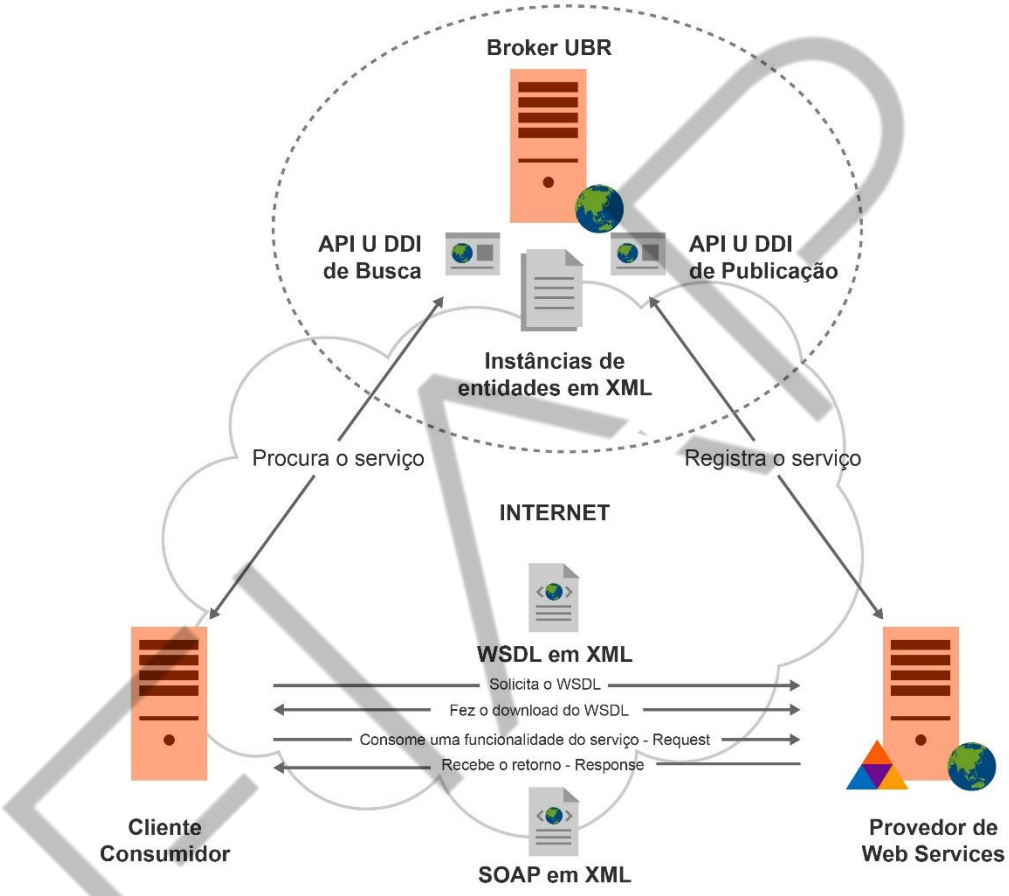

Figura 2.37 – Arquitetura de *Web Services* SOAP
Fonte: Elaborado pelo autor (2017)

O UDDI é o protocolo usado pelos *Brokers* para organizar e armazenar informações sobre serviços fornecidos na web. Ele possibilita ao *Broker* disponibilizar informações sobre organizações provedoras de serviço e seus respectivos *Web Services*, utilizando estruturas de dados em XML, chamadas entidades. Um modelo de informação UDDI, na versão 3.0.2, define que um cliente pode acessar as informações de provedores de serviços através de instâncias dos seguintes tipos de entidade XML:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- *businessEntity*: descreve uma empresa ou organização que fornece *Web Services*;
- *businessService*: descreve a coleção de *Web Services* oferecidos por uma organização (*businessEntity*);
- *bindingTemplate*: descreve as informações técnicas necessárias para usar um *Web Service* da coleção;
- *TModel*: descreve um "modelo técnico" que informa o tipo do *Web Service* e o protocolo usado pelo serviço;
- *publisherAssertion*: descreve a relação que a *businessEntity* possui com outra *businessEntity*;
- *subscription*: possibilita um pedido permanente para acompanhar as alterações que ocorram nas informações de uma *businessEntity*.

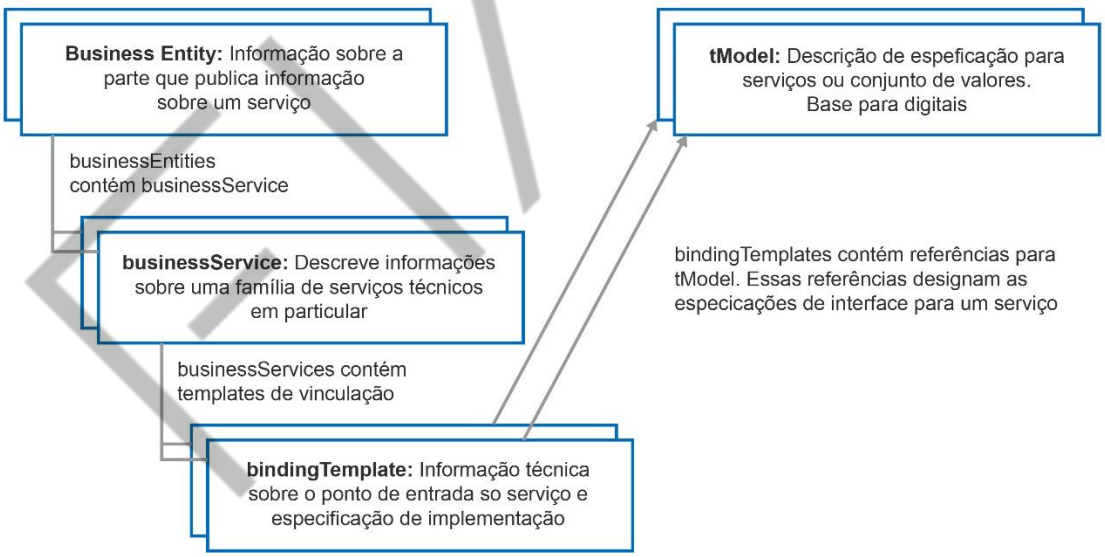

Figura 2.38 – UDDI *core data structures*
Fonte: OASIS, http://www.uddi.org/pubs/uddi-v3.0.2-20041019.htm

Por exemplo, informações importantíssimas para consumir um serviço (WSDL) são encontradas nas entidades mais técnicas, *bindingTemplate* e *tModel*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em 2006, empresas participantes das definições do UDDI, como a IBM, Microsoft e SAP avaliaram as respectivas utilizações da tecnologia UBR e resolveram descontinuar os seus projetos, fazendo o *shutdown* dos seus servidores UDDI.

As conversas aconteceram e ainda acontecem normalmente sem a necessidade de um *Broker*. O consumidor acaba acessando o WSDL diretamente do provedor, ou seja, a arquitetura envolvendo UBRs caiu em desuso, tal qual a utilização das históricas páginas amarelas.

Agora, vamos criar um exemplo de *Web Service* SOAP, sendo executado Tomcat e persistindo dados em um banco de dados Oracle. Vamos lá?

2.2.3 Exemplo 2 - *Web Service* com JAX-WS, CXF, JPA, Tomcat e Oracle

No Eclipse crie um novo *Dynamic Web Project:*

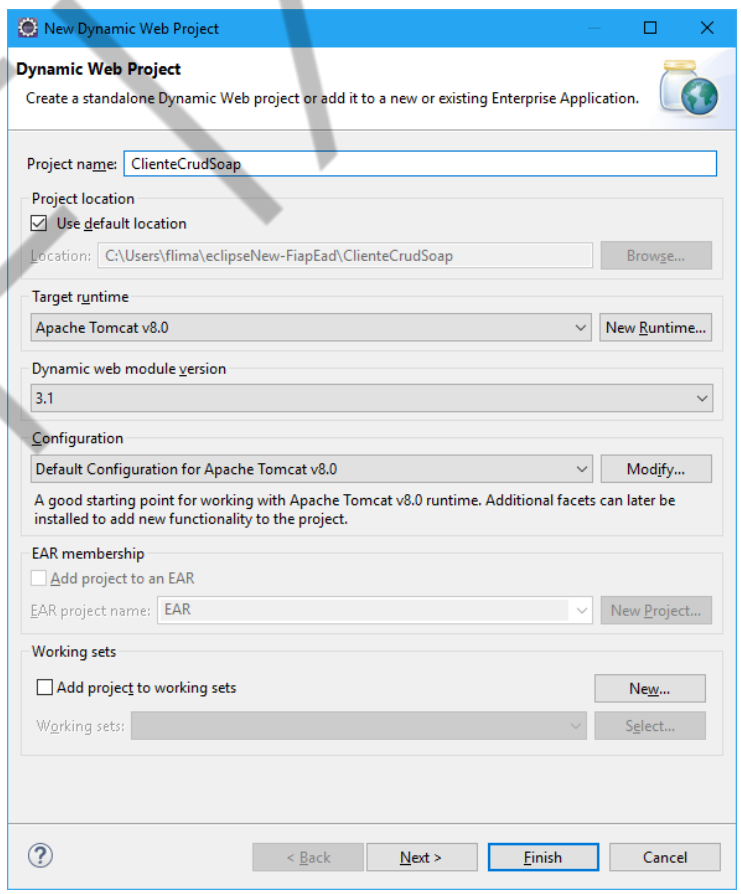

Figura 2.39 – Criando um *Dynamic Web Project* para o Exemplo 2

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Fonte: Elaborado pelo autor (2017)

Clique em *Next* na tela *Dynamic Web Project* e a tela abaixo aparecerá. Clique em *Next* novamente.

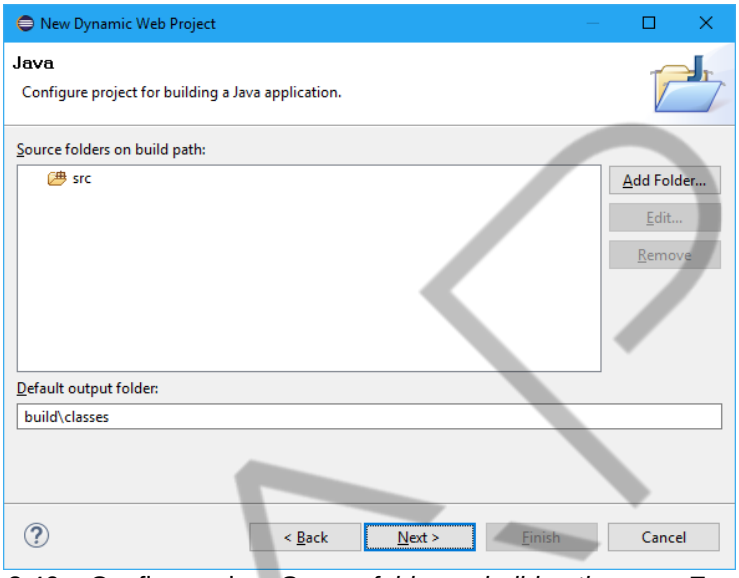

Figura 2.40 – Configurando o *Source folder* no *build path* para o Exemplo 2
Fonte: Elaborado pelo autor (2017)

Mantenha a configuração em *Configure web module settings* para gerar o arquivo web.xml e novamente clique em *Next*.

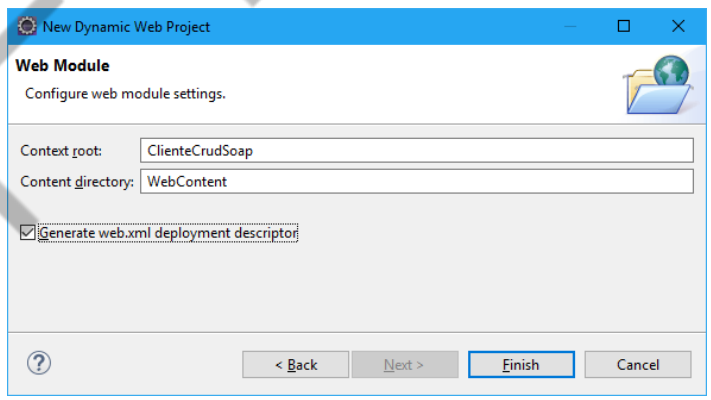

Figura 2.41 – Configurando o *web module* para o Exemplo 2
Fonte: Elaborado pelo autor (2017)

O projeto ClienteCrudSoap será criado com a seguinte estrutura:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

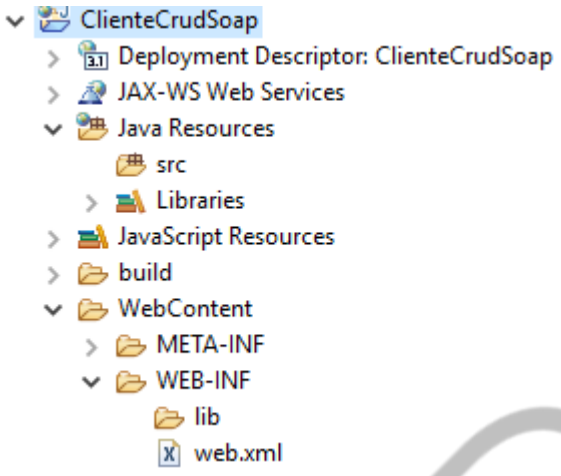

Figura 2.42 – Projeto ClienteCrudSoap no Projeto Explorer Eclipse do Exemplo 2
Fonte: Elaborado pelo autor (2017)

Adicione o jar do Oracle na pasta lib do seu projeto e adicione-o ao *Build Path* do projeto.

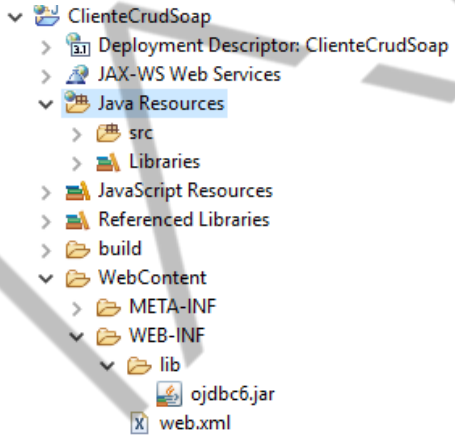

Figura 2.43 – Adicionando o jar Oracle no projeto ClienteCrudSoap do Exemplo 2.
Fonte: Elaborado pelo autor (2017)

Clique com o botão direito sobre o seu projeto e converta-o para Maven Project.

Figura 2.44 – Convertendo o projeto ClienteCrudSoap para Maven Project
Fonte: Elaborado pelo autor (2017)

Configure o POM do seu projeto e clique em *Finish*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

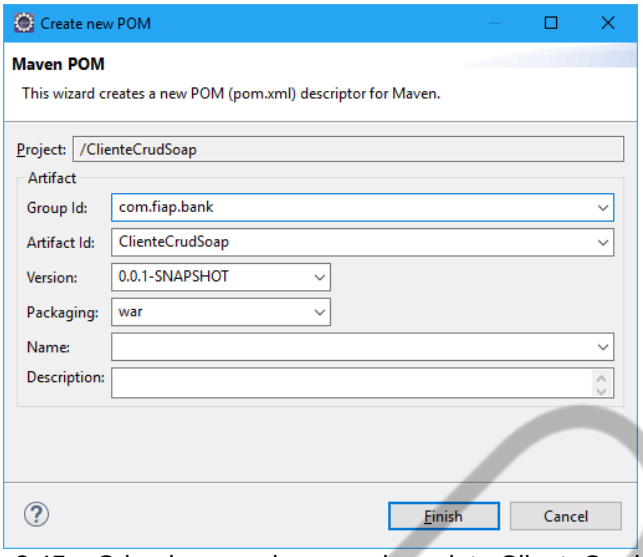

Figura 2.45 – Criando o arquivo pom do projeto ClienteCrudSoap.
Fonte: Elaborado pelo autor (2017)

No *Project Explorer* do Eclipse, clique no arquivo pom.xml gerado pelo *wizard* e adicione as propriedades e dependências necessárias para usarmos JPA e configurarmos um *pool* de conexão.

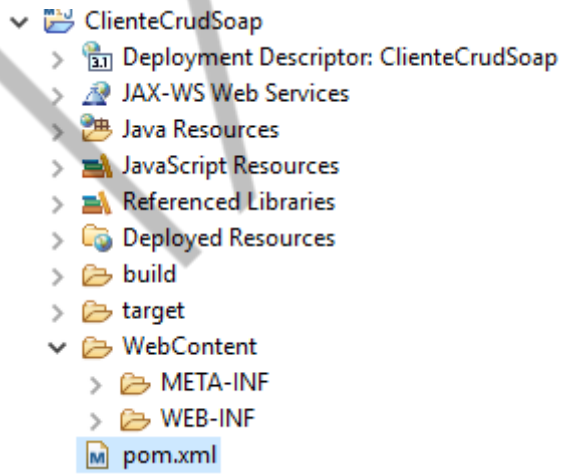

Figura 2.46 – O arquivo pom.xml do projeto ClienteCrudSoap no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Teremos duas exclusões na hierarquia do *Hibernate* configuradas no arquivo pom.xml para evitar conflitos com as bibliotecas do CXF.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  <groupId>com.fiap.bank</groupId>
  <artifactId>ClienteCrudSoap</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>


      <properties>
           <java.version>1.8</java.version>
           <hibernate.version>5.2.4.Final</hibernate.version>
           <project.build.sourceEncoding>
                UTF-8
     </project.build.sourceEncoding>
     </properties>

     <dependencies>

           <!-- JPA -->
           <dependency>
                <groupId>org.hibernate</groupId>
                <artifactId>hibernate-core</artifactId>
                <version>${hibernate.version}</version>
                <exclusions>
                     <exclusion>
                          <groupId>antlr</groupId>
                          <artifactId>antlr</artifactId>
                     </exclusion>
                     <exclusion>

     <groupId>org.apache.geronimo.specs</groupId>
                          <artifactId>
                                   geronimo-jta_1.1_spec
                               </artifactId>
                     </exclusion>
                </exclusions>
           </dependency>

           <dependency>
                <groupId>org.hibernate</groupId>
                <artifactId>hibernate-entitymanager</artifactId>
                <version>${hibernate.version}</version>
                <exclusions>
                     <exclusion>

     <groupId>org.apache.geronimo.specs</groupId>
                          <artifactId>geronimo-
jta_1.1_spec</artifactId>
                     </exclusion>
                </exclusions>
           </dependency>

           <!-- Para configuração do connection pooling -->
           <dependency>
                <groupId>org.hibernate</groupId>
                <artifactId>hibernate-c3p0</artifactId>
                <version>${hibernate.version}</version>
           </dependency>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      </dependencies>

  <build>
    <sourceDirectory>src</sourceDirectory>
    <plugins>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.6.1</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
      <plugin>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.0.0</version>
        <configuration>
          <warSourceDirectory>WebContent</warSourceDirectory>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Código-fonte 2.10 – pom.xml
Fonte: Elaborado pelo autor (2017)

Agora crie a pasta ClienteCrudSoap / Java Resources / src / META-INF e, nela, crie o arquivo persistence.xml com o seguinte código.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
      xmlns="http://xmlns.jcp.org/xml/ns/persistence"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence

      http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">
      <persistence-unit name="ORACLE_FIAP">

      <provider>org.hibernate.jpa.HibernatePersistenceProvider</pro
vider>
            <class>com.fiap.bank.entity.Cliente</class>
            <properties>
                  <property name="hibernate.show_sql" value="true"
/>
                  <property name="hibernate.format_sql" value="true"
/>
                  <property name="hibernate.hbm2ddl.auto"
value="update" />
                  <property name="hibernate.dialect"

value="org.hibernate.dialect.Oracle10gDialect" />
                  <property name="javax.persistence.jdbc.driver"
                              value="oracle.jdbc.OracleDriver" />
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                <property name="javax.persistence.jdbc.user"
value="PF0867" />
                <property name="javax.persistence.jdbc.password"
value="" />
                <property name="javax.persistence.jdbc.url"

value="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl" />

                <!-- Connection Pool -->
                <!-- Número mínimo de conexões  -->
                <property name="hibernate.c3p0.min_size" value="5"
/>
                <!-- Número máximo de conexões  -->
                <property name="hibernate.c3p0.max_size"
value="20" />
                <!-- Tempo de timeout, remoção da conexão do pool
-->
                <property name="hibernate.c3p0.timeout"
value="500" />
                <!-- Comandos em cache para aumento de performance
-->
                <property name="hibernate.c3p0.max_statements"
value="50" />
                <!-- Tempo para uma conexão ser validada
automaticamente -->
                <property name="hibernate.c3p0.idle_test_period"
                              value="2000" />
          </properties>
     </persistence-unit>
</persistence>
```

Código-fonte 2.11 – *persistence*.xml
Fonte: Elaborado pelo autor (2017)

Crie a classe *EntityManagerFactorySingleton* e implemente o código de acordo com o conhecimento adquirido na disciplina *Enterprise Application Development*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

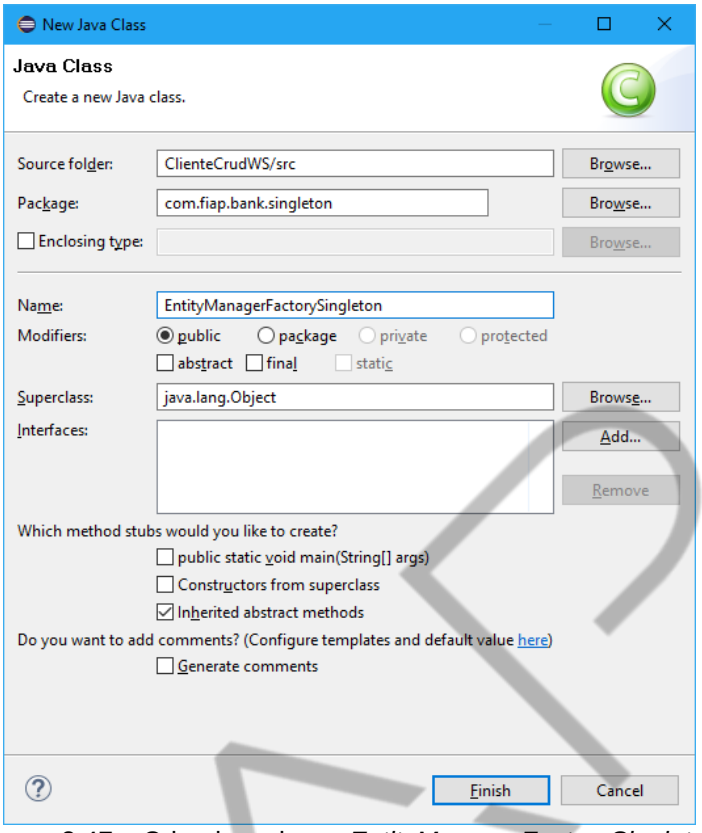

Figura 2.47 – Criando a classe *EntityManagerFactorySingleton*
Fonte: Elaborado pelo autor (2017)

```
package com.fiap.bank.singleton;

import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class EntityManagerFactorySingleton {

	private static EntityManagerFactory fabrica;

	private EntityManagerFactorySingleton(){}

	public static EntityManagerFactory getInstance(){
		if (fabrica == null){
			fabrica = Persistence
				.createEntityManagerFactory("ORACLE_FIAP");
		}
		return fabrica;
	}

}
```

Código-fonte 2.12 – EntityManagerFactorySingleton.java
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Acesse o Oracle FIAP e cria uma *sequence* de nome SEQ_CLIENTE.

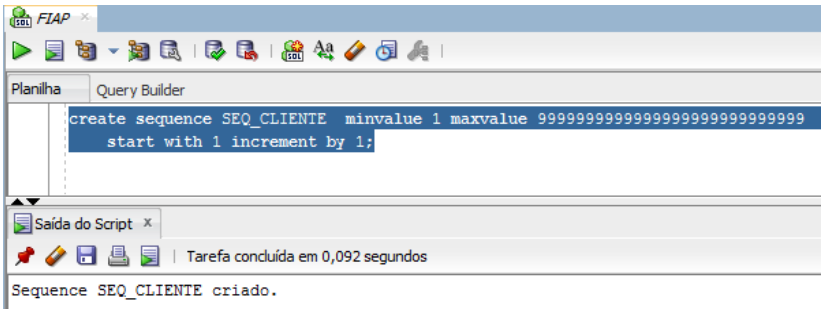

Figura 2.48 – Criando a *sequence* SEQ_CLIENTE no Oracle
Fonte: Elaborado pelo autor (2017)

Copie o pacote com.fiap.bank.entity do projeto OlaClienteWS para a pasta src do projeto atual, ClienteCrudSoap.

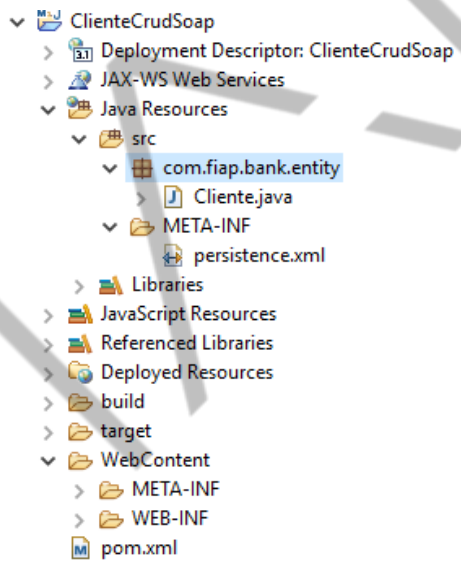

Figura 2.49 – O pacote com.fiap.bank.entity no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Altere a classe Cliente.java adicionando *annotations* e *imports,* necessários para a persistência JPA e *annotations* JAXB, pois essa entidade será exposta como XML pelo nosso Web Service.

```
package com.fiap.bank.entity;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import javax.persistence.Id;
import javax.persistence.SequenceGenerator;
import javax.persistence.Table;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;



@Entity
@Table(name="TBL_CLIENTE")
@SequenceGenerator(name="sqCliente", sequenceName="SEQ_CLIENTE",
allocationSize=1)
@XmlRootElement(name = "Cliente")
public class Cliente implements Serializable{

      private static final long serialVersionUID = 1L;

      @Id
      @GeneratedValue(strategy=GenerationType.SEQUENCE,generator="sqClien
te")
      @Column(name = "cd_cliente")
      private int idCliente;
      @Column(name = "nm_cliente", nullable = false, length = 40)
      private String nome;
      @Column(name = "ds_sobrenome", nullable = false, length = 60)
      private String sobrenome;
      @Column(name = "ds_email", nullable = false, length = 80)
      private String email;

      public Cliente(int idCliente, String nome, String sobrenome, String
email) {
            super();
            this.idCliente = idCliente;
            this.nome = nome;
            this.sobrenome = sobrenome;
            this.email = email;
      }

      public Cliente(){}

      @XmlElement(name = "idCliente", required = true)
      public int getIdCliente() {
            return idCliente;
      }
      public void setIdCliente(int idCliente) {
            this.idCliente = idCliente;
      }

      @XmlElement(name = "nome ", required = true)
      public String getNome() {
            return nome;
      }
      public void setNome(String nome) {
            this.nome = nome;
      }

      @XmlElement(name = "sobrenome", required = true)
      public String getSobrenome() {
            return sobrenome;
      }
      public void setSobrenome(String sobrenome) {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            this.sobrenome = sobrenome;
      }

      @XmlElement(name = "email", required = true)
      public String getEmail() {
            return email;
      }
      public void setEmail(String email) {
            this.email = email;
      }

      @Override
      public String toString() {
            return "Cliente [idCliente="
                                          + idCliente
                                          + ", nome="
                                          + nome
                                          + ", sobrenome="
                                          + sobrenome
                                          + ", email="
                                          + email
                                          + "]";
      }

}
```

Código-fonte 2.13 – Cliente.java
Fonte: Elaborado pelo autor (2017)

Crie uma classe Teste no pacote com.fiap.bank.test e execute-a, para testar a persistência da classe Cliente no Oracle FIAP.

```
package com.fiap.bank.test;

import javax.persistence.EntityManager;
import com.fiap.bank.entity.Cliente;
import com.fiap.bank.singleton.EntityManagerFactorySingleton;

public class Teste {

      public static void main(String[] args) {

            EntityManager em = null;

            Cliente cliente = new Cliente();
            cliente.setNome("Fernando");
            cliente.setSobrenome("Gouveia Lima");
            cliente.setEmail("qualquer@gmail.com");

            try {
                  em = EntityManagerFactorySingleton
                            .getInstance()
                            .createEntityManager();
                  em.getTransaction().begin();
                  em.persist(cliente);
                  em.getTransaction().commit();
            } catch (Exception e) {
                  if (em.getTransaction().isActive()) {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                        em.getTransaction().rollback();
                    }
                    e.printStackTrace();
                    System.out.println("Cadastro não realizado");
                } finally {
                    em.close();
                }
        }
}
```

Código-fonte 2.14 – Teste.java
Fonte: Elaborado pelo autor (2017)

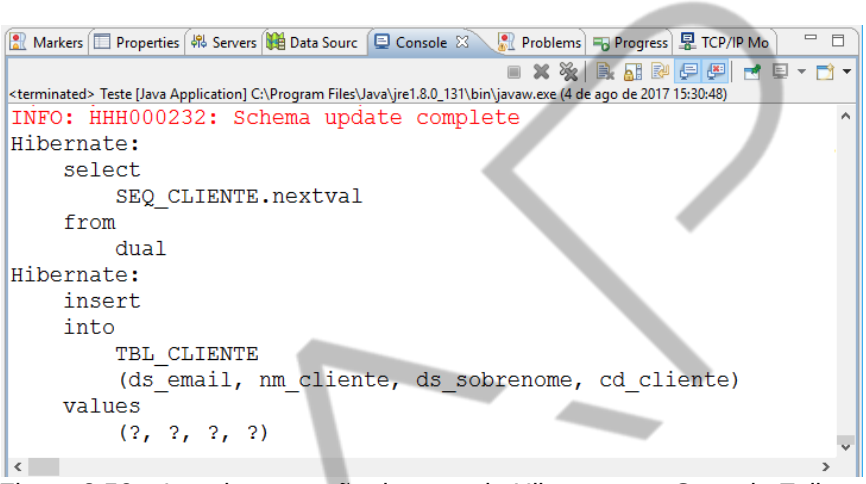

Figura 2.50 – Log de execução de teste do Hibernate no Console Eclipse
Fonte: Elaborado pelo autor (2017)

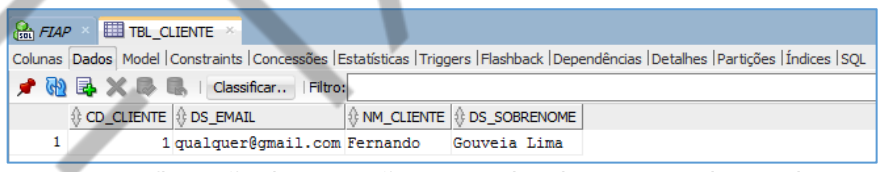

Figura 2.51 – Confirmação de gravação no Oracle, via acesso pelo Oracle Developer
Fonte: Elaborado pelo autor (2017)

Teste bem-sucedido, vamos criar a camada DAO do nosso serviço. Para tanto, vamos utilizar as classes *DataBaseException*, *EntityNotFoundException*, *GenericDAO* e *GenericDAOImpl* que você desenvolveu na disciplina *Enterprise Application Development*. Crie os pacotes e as classes conforme a imagem abaixo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

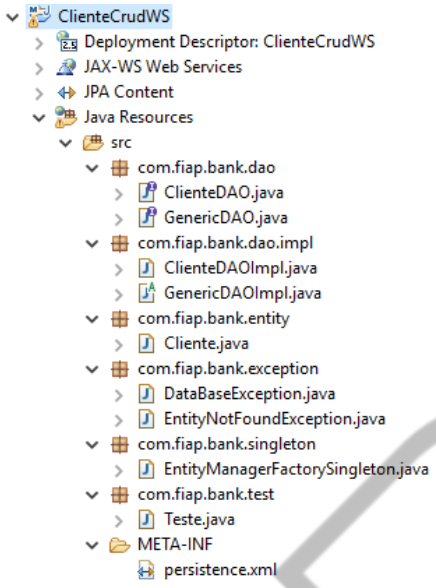

Figura 2.52 – Criação de pacotes de persistência no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

Desenvolva a interface *ClienteDAO* e a classe *ClienteDAOImpl* conforme os seguintes códigos:

```
package com.fiap.bank.dao;

import com.fiap.bank.entity.Cliente;

public interface ClienteDAO
      extends GenericDAO<Cliente, Integer>{

}
```

Código Fonte 2.15 – ClienteDAO.java
Fonte: Elaborado pelo autor (2017)

```
package com.fiap.bank.dao.impl;

import com.fiap.bank.dao.ClienteDAO;
import com.fiap.bank.entity.Cliente;

public class ClienteDAOImpl
      extends GenericDAOImpl<Cliente, Integer>
      implements ClienteDAO{

}
```

Código-fonte 2.16 – ClienteDaoImpl.java
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Crie uma classe *ClienteBO* no pacote com.fiap.bank.bo e codifique-a, conforme trecho abaixo:

```
package com.fiap.bank.bo;

import com.fiap.bank.dao.ClienteDAO;
import com.fiap.bank.dao.impl.ClienteDAOImpl;
import com.fiap.bank.entity.Cliente;
import com.fiap.bank.exception.DataBaseException;

public class ClienteBO {

   private ClienteDAO dao = new ClienteDAOImpl();

   public Cliente buscarPorId(Integer idCliente){
      Cliente Cliente = dao.buscarPorId(idCliente);
      return Cliente;
   }

   public Cliente adicionar(Cliente cliente) {
      Cliente cli = null;
      try {
         cli = dao.adicionar(cliente);
      } catch (DataBaseException de) {
         de.printStackTrace();
      }
      return cli;
   }
}
```

Agora, vamos codificar a implementação de um *Web Service* CXF para expor as duas funcionalidades da nossa classe *ClienteBO*. Crie uma classe *ClienteServiceImpl* no pacote com.fiap.bank.ws e desenvolva-a, conforme código abaixo.

```
package com.fiap.bank.ws;

import com.fiap.bank.bo.ClienteBO;
import com.fiap.bank.entity.Cliente;

public class ClienteServiceImpl {

      ClienteBO ClienteBO = new ClienteBO();

      public Cliente buscarPorId(Integer idCliente){
            return ClienteBO.buscarPorId(idCliente);
      }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        public void cadastrarCliente(Cliente cliente){
              ClienteBO.adicionar(cliente);
        }
}
```

Código-fonte 2.17 – ClienteServiceImpl.java
Fonte: Elaborado pelo autor (2017)

A interface lógica, definida pelo elemento wsdl: portType, é mapeada para uma interface de ponto de extremidade de serviço (SEI). Todos os tipos complexos definidos no WSDL são mapeados em classes Java seguindo o mapeamento definido pela especificação *Java Architecture for XML Binding* (JAXB).

Com a implementação criada, já temos o suficiente para gerar o *Web Service* utilizando o *wizard* Eclipse e o CXF Runtime 3.1.12 que configuramos em atividade anterior. Vamos lá?

Clique com o botão direito em na classe *ClienteServiceImpl*, navegue em *New > Other > Web Services* e clique em *Next*.

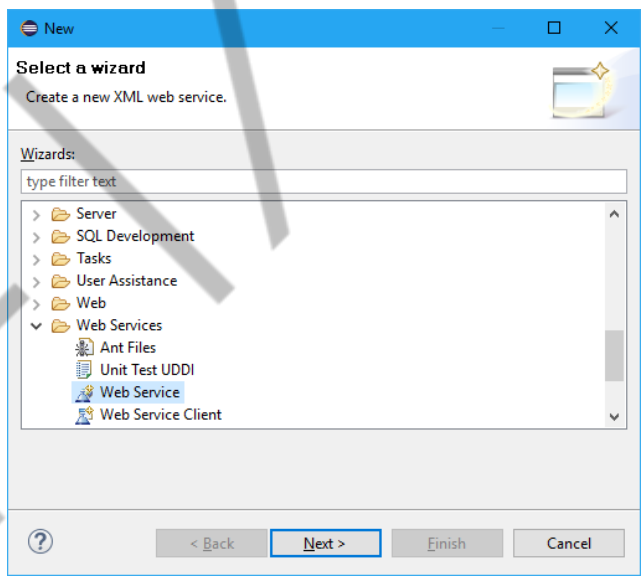

Figura 2.53 – Geração de um *Web Service* no wizard do Eclipse
Fonte: Elaborado pelo autor (2017).

Mova o slider para o nível *Develop service* e garanta a configuração do *Service implementation*, Tomcat, *runtime* Apache CXF 2.x e *Service project*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

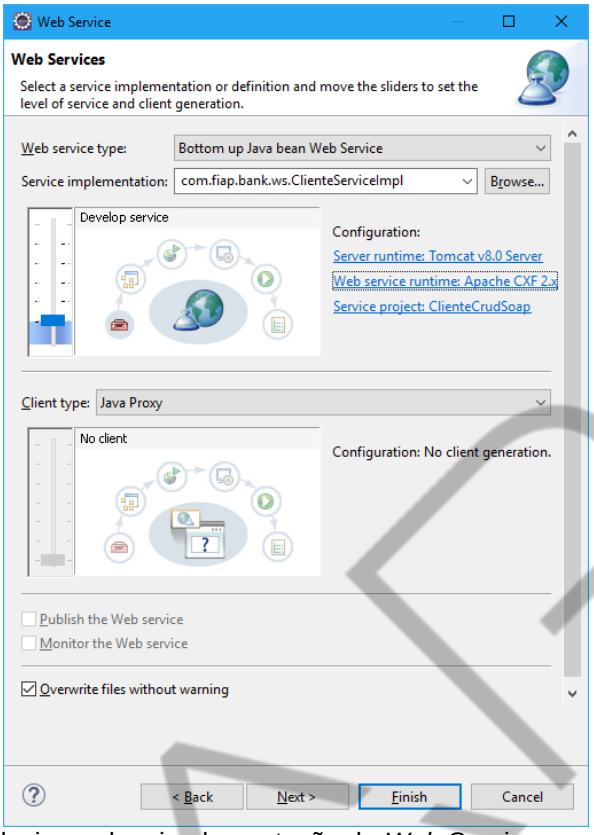

Figura 2.54 – Selecionando a implementação do *Web Service* no *wizard* do Eclipse
Fonte: Elaborado pelo autor (2017)

Configure o *wizard* para que ele crie um *Service Endpoint Interface* (SEI) com base na classe *ClienteServiceImpl* e clique em *Next*.

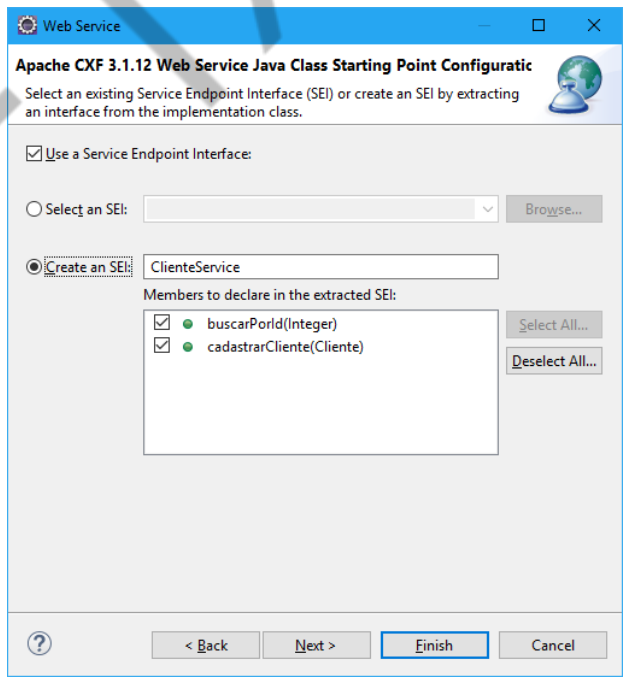

Figura 2.55 – Configurando um SEI Apache CXF no *wizard* do Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Não configure nenhuma anotação adicional, clique em *Next*.

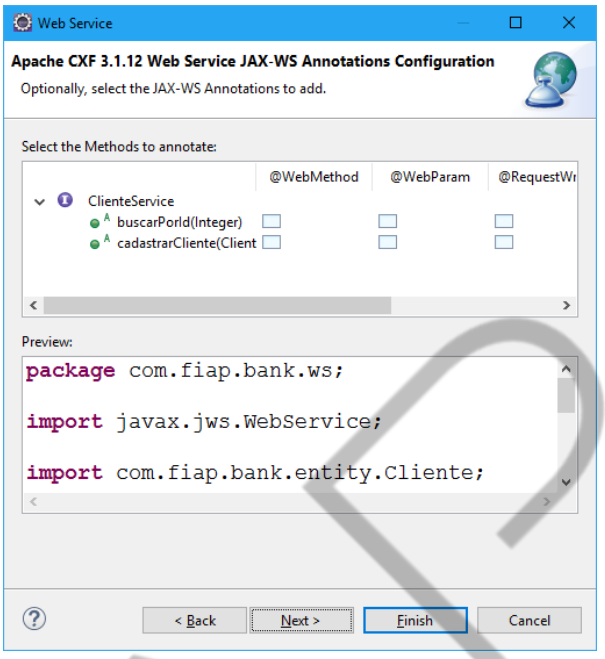

Figura 2.56 – Configurando anotações JAX-WS opcionais no *wizard* do Eclipse
Fonte: Elaborado pelo autor (2017)

Configure o *wizard* do Eclipse para criar o WSDL do seu serviço e clique em *Finish*. O JAX-WS especificará um mapeamento detalhado do *Web Service* no WSDL, para a classe *ClienteServiceImpl* que implementará o serviço. A interface lógica, definida pelo elemento wsdl: portType, será mapeada para o (SEI) que criamos no passo anterior.

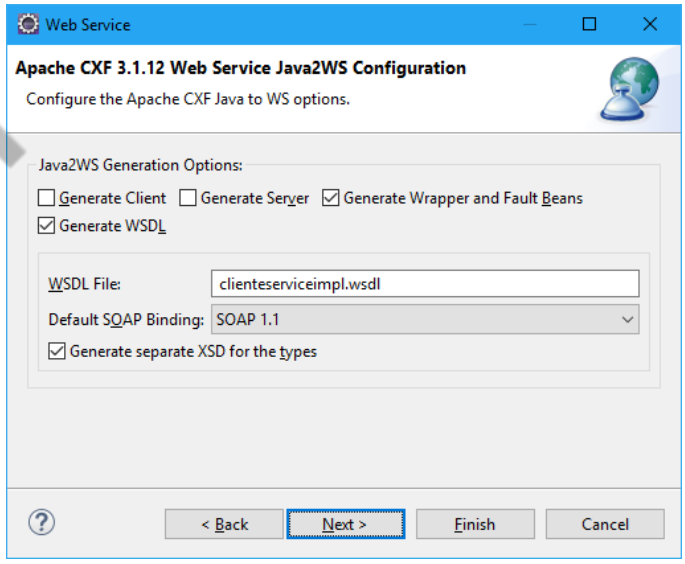

Figura 2.57 – Configurando Java2WS no *wizard* do Eclipse
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O *wizard* do Eclipse e a biblioteca Apache CXF 3.1.12 criarão o SEI, o WSDL e as classes necessárias para que o JAXWS disponibilize os métodos buscarPorId e cadastrarCliente.

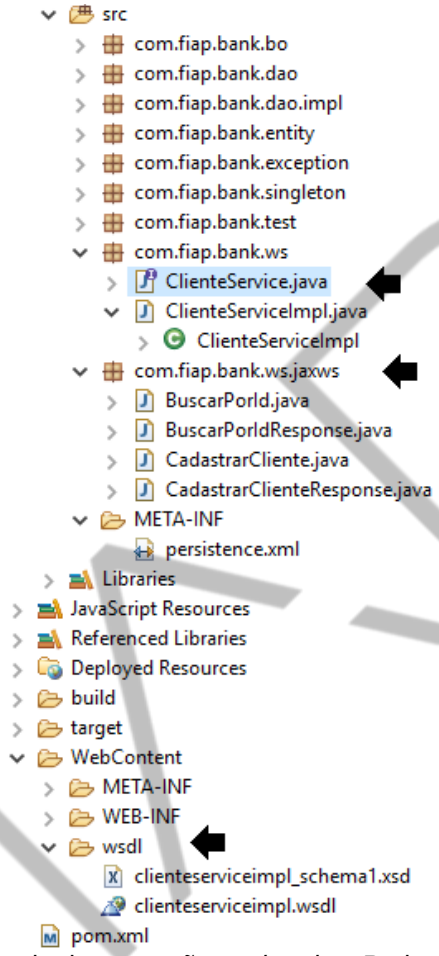

Figura 2.58 – Resultado da execução o *wizard* no Project Explorer do Eclipse
Fonte: Elaborado pelo autor (2017)

O SEI gerado em com.fiap.bank será assim:

```
package com.fiap.bank.ws;

import javax.jws.WebService;

import com.fiap.bank.entity.Cliente;

@WebService( name = "ClienteService",
             targetNamespace = "http://ws.bank.fiap.com/")


public interface ClienteService {

     Cliente buscarPorId(Integer idCliente);
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      Cliente cadastrarCliente(Cliente cliente);

}
```

Código-fonte 2.18 – ClienteService.java
Fonte: Elaborado pelo autor (2017)

A anotação @WebService é definida pela interface javax.jws.WebService e é colocada na nossa interface para ela ser usada como um serviço.

A propriedade *name* do @WebSerivice especifica o nome da interface do serviço. Essa propriedade é mapeada para o atributo de *name* do elemento wsdl: portType que define a interface do serviço em um contrato WSDL.

A propriedade *targetNameSpace* especifica o *namespace* destino, sob o qual o serviço foi definido. O *namespace* foi derivado do nome do pacote

A classe de implementação *ClienteServiceImpl* sofrerá alteração pelo *wizard* e ficará com o seguinte código.

```
package com.fiap.bank.ws;

import javax.jws.WebService;
import com.fiap.bank.bo.ClienteBO;
import com.fiap.bank.entity.Cliente;

@WebService(targetNamespace = "http://ws.bank.fiap.com/",
endpointInterface = "com.fiap.bank.ws.ClienteService", portName =
"ClienteServiceImplPort", serviceName = "ClienteServiceImplService")
public class ClienteServiceImpl implements ClienteService{

      ClienteBO ClienteBO = new ClienteBO();

      public Cliente buscarPorId(Integer idCliente){
            return ClienteBO.buscarPorId(idCliente);
      }

      public Cliente cadastrarCliente(Cliente cliente){
            return ClienteBO.adicionar(cliente);
      }
}
```

Código-fonte 2.19 – ClienteServiceImpl.java
Fonte: Elaborado pelo autor (2017)

Além de anotar o SEI com a anotação @WebService, o *wizard* também anotou a classe de implementação do serviço com @WebService. Ao adicionar a anotação à classe de implementação do serviço, o gerador de código também especificou a propriedade *endpointInterface*, com o nome completo do SEI.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A propriedade *targetNameSpace*, como já vimos, especifica o *namespace* destino, sob o qual o serviço foi definido.

A propriedade *portName* especifica o nome do *endpoint* no qual o serviço será publicado. Essa propriedade é mapeada para o atributo *name* do wsdl:port element do WSDL que especifica os detalhes do *endpoint* para o serviço quando publicado. O padrão é manter o nome da classe de implementação do serviço.

A propriedade *serviceName* especifica o nome do serviço ao ser publicado. Essa propriedade é mapeada para o atributo de *name* do wsdl:service element que define o serviço publicado no WSDL.

O *wizard* também alterará o arquivo web.xml que deverá ficar assim, após removermos o elemento <welcome-file-list>...</welcome-file-list> e seu conteúdo:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xmlns="http://xmlns.jcp.org/xml/ns/javaee"
           xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee

      http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
      id="WebApp_ID" version="3.1">
  <display-name>ClienteCrudSoap</display-name>
  <servlet>
    <description>Apache CXF Endpoint</description>
    <display-name>cxf</display-name>
    <servlet-name>cxf</servlet-name>
    <servlet-class>
      org.apache.cxf.transport.servlet.CXFServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>cxf</servlet-name>
    <url-pattern>/services/*</url-pattern>
  </servlet-mapping>
  <session-config>
    <session-timeout>60</session-timeout>
  </session-config>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>WEB-INF/cxf-beans.xml</param-value>
  </context-param>
  <listener>
    <listener-class>
      org.springframework.web.context.ContextLoaderListener
    </listener-class>
  </listener>
</web-app>
```

Código-fonte 2.20 – web.xml
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O elemento <servlet> declara o Servlet CXF baseado no *Spring CXFServlet* que será usado para disponibilizar o *Web Service* que desenvolvemos. Para conhecer o *Web Service* implantado, acessamos o Tomcat em http://localhost:8088/ClienteCrudSoap/services, definido em localhost:<port>/<display-name>/<url-pattern> no web.xml. Eu configurei a minha porta para 8088, você deve estar utilizando a default 8080.

O *servlet* usa o *Spring ApplicationContext* carregado a partir do arquivo cxf-beans.xml definido no <context-param> do web.xml. O <listener-class> é usado para integrar o Spring, em nosso exemplo, com o Tomcat.

Vejamos o arquivo cxf-beans.xml. Precisamos fazer uma pequena alteração nele. Remova a linha em vermelho, apresentada no quadro abaixo. Essa ação é necessária, pois a versão do Apache CXF 3.1.12 não necessita mais o cxf-extension-soap.xml. Isso evitará um erro na publicação do nosso *Web Service* no Tomcat.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:jaxws="http://cxf.apache.org/jaxws"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.5.xsd
http://cxf.apache.org/jaxws http://cxf.apache.org/schemas/jaxws.xsd">
       <import resource="classpath:META-INF/cxf/cxf.xml" />
       <import resource="classpath:META-INF/cxf/cxf-extension-soap.xml" />
       <import resource="classpath:META-INF/cxf/cxf-servlet.xml" />
       <jaxws:endpoint xmlns:tns="http://ws.bank.fiap.com/" id="clienteservice"
              implementor="com.fiap.bank.ws.ClienteServiceImpl"
wsdlLocation="wsdl/clienteserviceimpl.wsdl"
              endpointName="tns:ClienteServiceImplPort"
serviceName="tns:ClienteServiceImplService"
              address="/ClienteServiceImplPort">
              <jaxws:features>
                     <bean class="org.apache.cxf.feature.LoggingFeature" />
              </jaxws:features>
       </jaxws:endpoint>
</beans>
```

Código-fonte 2.21 – cxf-beans.xml
Fonte: Elaborado pelo autor (2017)

Vamos olhar os pontos principais desse arquivo de configuração.

`implementor="com.fiap.bank.ws.ClienteServiceImpl"` publica a classe *ClienteServiceImpl* que implementa o nosso serviço;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

`endpointName="tns:ClienteServiceImplPort"` define a URL na qual o nosso serviço será publicado. Ou seja, http://localhost:8088/ClienteCrudSoap/services/*ClienteServiceImplPort* (Não acessível, sem o envio de um envelope SOAP de requisição).

`wsdlLocation="wsdl/clienteserviceimpl.wsdl"` define a URL na qual você pode realizar o download do WSDL no nosso serviço CXF. Para acessá-lo, busque no browser por: http://localhost:8088/ClienteCrudSoap/wsdl/clienteserviceimpl.wsdl ou pelo *endpoint* definido em `endpointName="tns:ClienteServiceImplPort"`, acessando http://localhost:8088/ClienteCrudSoap/services/ClienteServiceImplPort?wsdl no browser.

Agora, vamos testar o nosso *Web Service* com o SoapUI?

Crie um novo projeto SOAP:

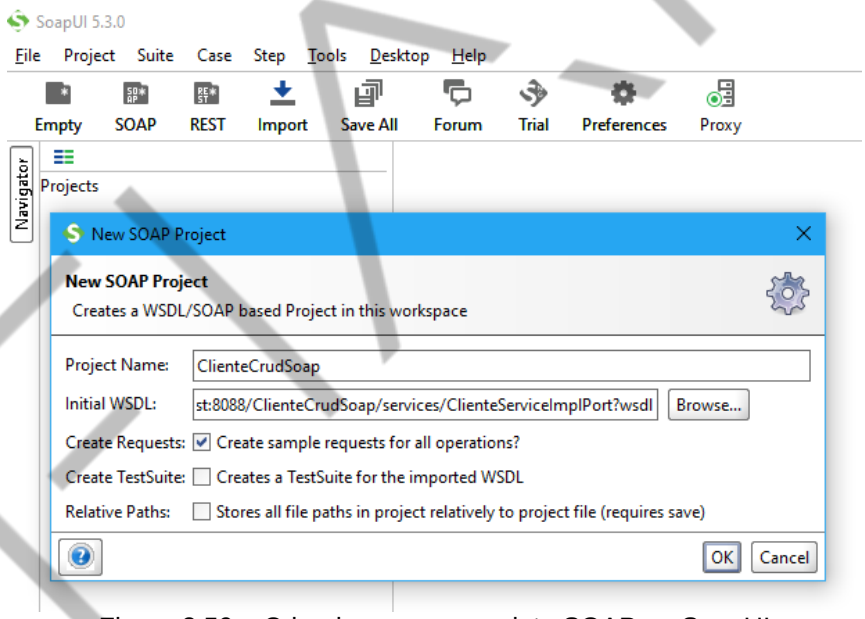

Figura 2.59 – Criando um novo projeto SOAP no SoapUI
Fonte: Elaborado pelo autor (2017)

Utilize o método cadastrarCliente, preencha os parâmetros do *Request* 1 no Envelope SOAP e execute a requisição. O nosso *Web Service* cadastrará o cliente no Oracle e retornará o cliente cadastrado, para que tenhamos acesso ao id gerado pela *sequence* no banco.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

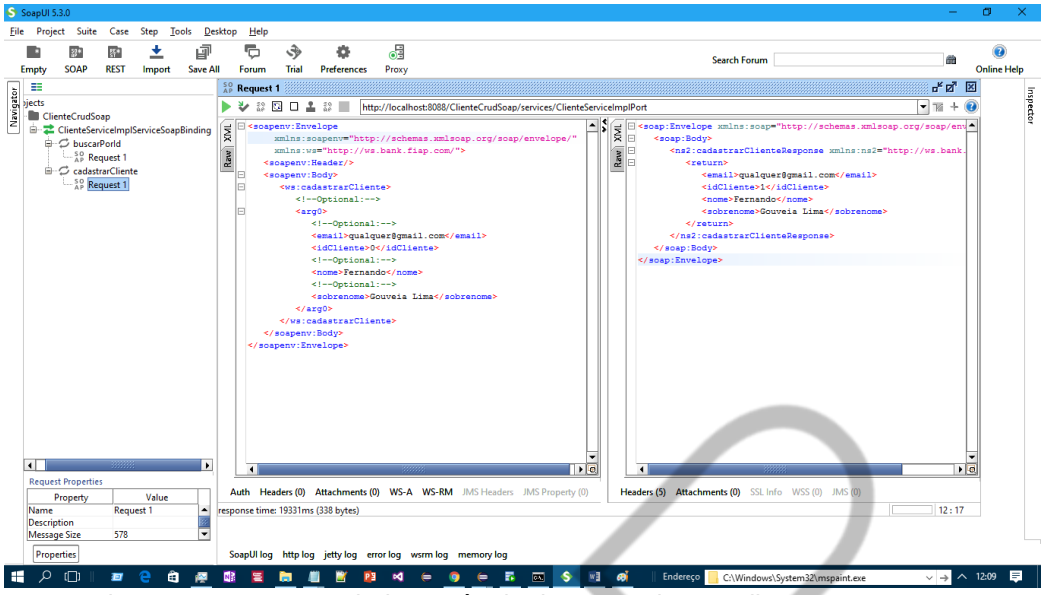

Figura 2.60 – Consumindo o método do WS cadastrarCliente no SoapUI.
Fonte: Elaborado pelo autor (2017)

Agora vamos testar o método buscaPorId. Como fizemos no cadastro, preencha os parâmetros do *Request* 1 no Envelope SOAP e execute a requisição. O nosso *Web Service* retorna o cliente cadastrado no Oracle conforme o id requisitado.

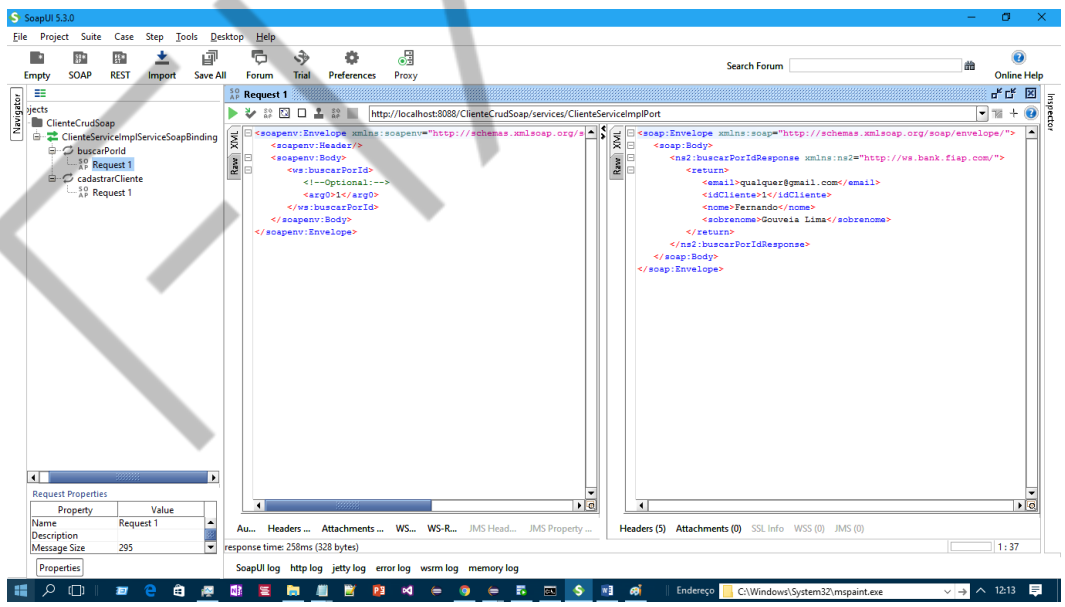

Figura 2.61 – Consumindo o método do WS buscarPorId no SoapUI
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Considerações Finais

O objetivo deste conteúdo foi abordar o básico e essencial sobre SOA, *"Big" Web Services*, SOAP e Apache CXF. Sendo assim, existem diversos conteúdos que são abordados nos livros e URLs de referência deste capítulo. Para mais informações, recomendamos a leitura.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ALVARENGA , Darlan. **Mobile banking se torna meio mais usado para transações bancárias, diz Febraban**. Publicado em: 10 mai. 2017. Disponível em: <http://g1.globo.com/economia/seu-dinheiro/noticia/mobile-banking-se-torna-meio-mais-usado-para-transacoes-bancarias-diz-febraban.ghtml>. Acesso em: 28 jun. 2017.

APACHE SOFTWARE FOUNDATION. **Developing a Service.** Disponível em: http://cxf.apache.org/docs/developing-a-service.html. Acesso em: 04 jul. 2017.

_____. **Apache CXF™: An Open-Source Services Framework**. Disponível em: <http://cxf.apache.org/>. Acesso em: 19 jul. 2017.

BOOTH David; HAAS, Hugo; MCCABE, Francis; NEWCOMER, Eric; CHAMPION Michael; FERRIS Chris; ORCHARD, David. **Web Services Architecture Note 11 February 2004.** Disponível em: <https://www.w3.org/TR/ws-arch/>. Acesso em: 28 jun. 2017.

CLEMENT, Luc; HATELY, Andrew; VON RIEGEN, Claus; ROGERS, Tony. **UDDI Spec TC**. Disponível em: http://www.uddi.org/pubs/uddi-v3.0.2-20041019.htm. Acesso em: 30 jul. 2017.

GOMES, Daniel Adorno. **Web services SOAP em Java**. São Paulo: Novatec, 2010.

GONÇALVES, Antônio. **Introdução à Plataforma Java EE6 com GlassFish**. Rio de Janeiro: Editora Ciência Moderna LTDA, 2011.

HOELLER, Juerge; BEAMS, Chris. **Spring Framework 4.3.10.RELEASE API**. Disponível em: https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/web/context/ContextLoaderListener.html. Acesso em: 05 jul. 2017.

ISTOÉ. **Internet banking já é mais usado do que agências no Brasil**. 29/10/13 - 15h07 - Atualizado em 21/01/16 - 12h43. Disponível em: <http://istoe.com.br/332073_INTERNET+BANKING+JA+E+MAIS+USADO+DO+QUE+AGENCIAS+NO+BRASIL/>. Acesso em: 28 jun. 2017.

JOSUTTIS, Nicolai M. **SOA na Prática**. Rio de Janeiro: Alta, 2008.

KRESS, Jürgen; MAIER, Berthold; NORMANN, Hajo; SCHMEIDEL, Danilo; SCHMUTZ Guido; TROPS, Bernd; UTSCHIG-UTSCHIG, Clemens; WINTERBERG Torsten. **Service-Oriented Architecture**. Oracle Technology Network. Publicado em: Jul. 2013. Disponível em: http://www.oracle.com/technetwork/articles/soa/ind-soa-esb-1967705.html. Acesso em: 27 jun. 2017.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LEGATTI, Eduardo. **Coluna do tipo autoincremento no Oracle? Abordando o uso de sequências no Oracle.** Disponível em:

http://www.oracle.com/technetwork/pt/articles/database-performance/abordando-o-uso-sequencias-oracle-495861-ptb.html. Acesso em: 04 jul. 2017.

ORACLE. **Fusion Middleware Concepts and Architecture for Oracle Service Bus.** Disponível em: <https://docs.oracle.com/cd/E21764_01/doc.1111/e15020/introduction.htm#OSBCA107>. Acesso em: 30 jun. 2017.

_____. **The Java EE 6 Tutorial. 2013**. Disponível em: <http://docs.oracle.com/javaee/6/tutorial/doc/giqsx.html >. Acesso em: 30 jun. 2017.

SAMPAIO, CLEUTON. **Java Enterprise Edition 6-Desenvolvendo Aplicações Corporativas**. Brasport, 2011.

SPROTT David; WILKES, Lawrence. **Understanding Service-Oriented Architecture.** Disponível em: https://msdn.microsoft.com/en-us/library/aa480021.aspx. Acesso em: 30 jun. 2017.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com